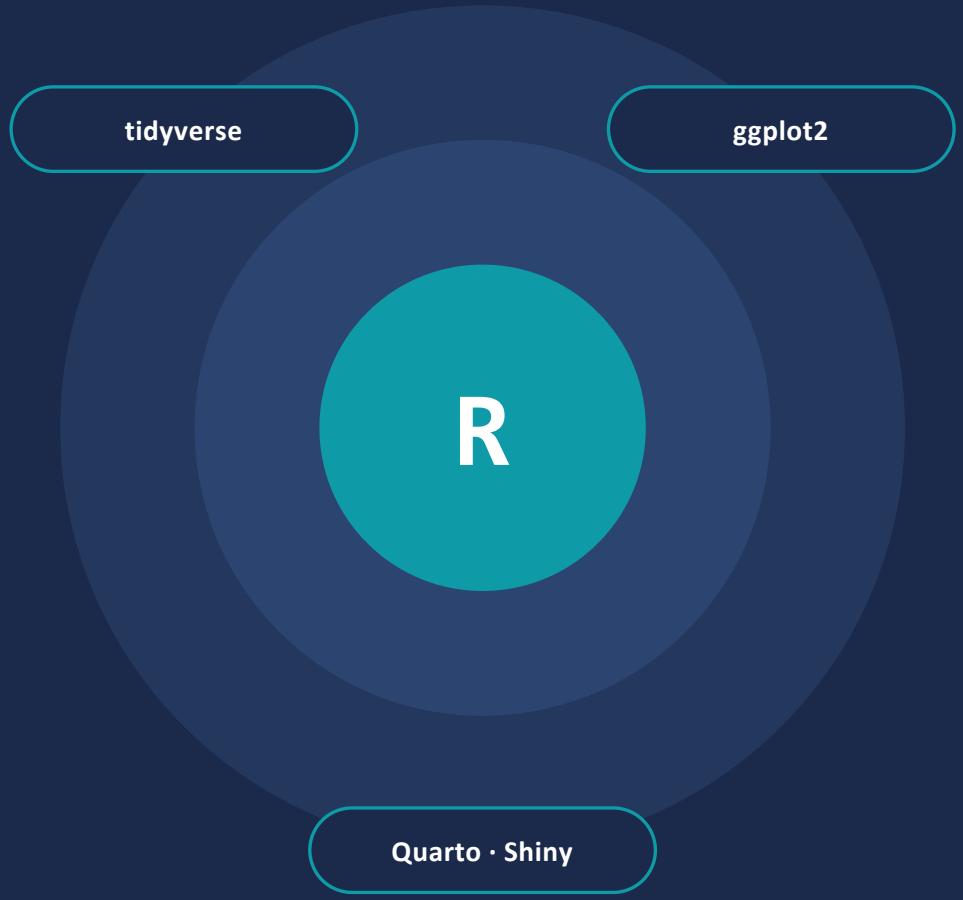
TRAINING PROGRAMME

Data Analysis with R

A three-day, hands-on training programme for ESS Senior Experts — from R fundamentals to interactive dashboards.

18 – 20 September 2026

Tony Blair Institute for Global Change
in partnership with the Ethiopian Statistical Service (ESS)



Core competency areas of this training

Five competencies, built progressively over three days using ESS livestock and crop datasets.

1 R programming fundamentals

Apply R syntax, data types, operators, functions and control structures with confidence.

2 Data manipulation & preparation

Import, clean, transform, join and organise datasets with base R, dplyr and tidyr.

3 Exploratory data analysis (EDA)

Summarise and explore data to reveal patterns, trends, relationships and anomalies.

4 Data visualisation

Create professional, informative graphics with ggplot2 and Plotly.

5 Dashboard development

Build interactive analytical dashboards with Quarto and Shiny.

Expected training outcomes

- By the end of the training, participants will be able to:
  - Import, clean, transform and manage real-world datasets using R.
  - Perform descriptive and exploratory data analysis independently.
  - Create high-quality static and interactive data visualisations.
  - Develop interactive dashboards for monitoring and communicating key indicators.
  - Translate analytical results into clear, actionable insights for evidence-based decision-making.



DISCUSSION

Which of these outcomes matters most for your current role at ESS? Share one example of a task it would help with.

Training at a glance: key dates

Three consecutive days, 08:30 – 17:00. Each day has two tea breaks and a lunch break.

1 2 3

DAY 1 · FRIDAY

18 September 2026

R, RStudio & the fundamentals of the language

- RStudio, scripts and help
- Data types, objects, operators
- Vectors, matrices, data frames
- Conditions, loops, functions

DAY 2 · SATURDAY

19 September 2026

Importing, exploring and preparing data

- Packages and reading files
- Summaries and frequency tables
- Subsetting and missing values
- dplyr verbs and merging

DAY 3 · SUNDAY

20 September 2026

Visualisation, reporting & dashboards

- ggplot2 grammar of graphics
- Scatter, line and bar charts
- R Markdown and Quarto
- Quarto & Shiny dashboards, post-assessment

How we will work together

A few ground rules to make the three days productive.

1 Learn by doing

Every concept is followed by live coding on ESS datasets. Type along — do not just watch.

2 Ask early

If a command fails, raise it immediately. Error messages are the fastest way to learn.

3 Pair up

Sit with a colleague; help each other debug before calling a facilitator.

4 Reflect daily

We close each day with a short reflection and open the next with a recap.



FACILITATOR TIP

Confirm before Module 1 that every laptop has R and RStudio installed, plus the tidyverse, readxl and haven packages.



QUICK CHECK

25 min

Pre-assessment: where are we starting from?

1. Fifteen short open questions covering everything from R basics to dashboards.

2. Answer individually in one or two sentences each — this is not graded. It helps facilitators pace the training and lets you measure your own progress on Day 3.

3. Write your answers on the response sheet provided (Q1–Q15). Leave blank anything you do not know yet.

How to run it: Show each slide for about 5 minutes. Collect the sheets; keep them for comparison with the post-assessment on Day 3.

Pre-assessment

Q1 **In your own words, what is R and what is RStudio? How do the two relate to each other?**

Q2 **Why does an analyst need a *script* rather than typing commands one by one in the console?**

Q3 **Explain the difference between a numeric, a character, a logical and a factor value in R. Give one example of each from survey data.**

Pre-assessment

Q4 What does the assignment operator `<-` do? Write one line that stores the value 2016 in an object called `year`.

Q5 A colleague has a vector of region codes 1, 2, 3. How would you attach the labels 'Afar', 'Amhara', 'Oromia' to them?

Q6 Describe what a data frame is and how it differs from a matrix.

Pre-assessment

Q7 Write, in words or code, how you would read a CSV file named `livestock.csv` into R.

Q8 What do `head()`, `str()` and `summary()` tell you about a dataset? When would you use each?

Q9 A column contains missing values. What happens when you call `mean()` on it, and how do you fix it?

Pre-assessment

Q10 How would you keep only the rows of a data frame where `Region` equals 'Oromiya' and `year` is greater than 2016?

Q11 Name three dplyr verbs and describe what each one does.

Q12 Explain what the pipe `%>%` does and why analysts like it.

Pre-assessment

Q13 In ggplot2, what is an *aesthetic* and what is a *geom*? Give an example of each.

Q14 You have two files that share a `zone` column. How would you combine them into one table, and what could go wrong?

Q15 What is a dashboard, and what is the difference between a report and a dashboard?

TRAINING DAY

1

Friday, 18 September 2026

R, RStudio and the fundamentals of the language

Today's schedule

08:30 – 09:00	Registration, welcome & pre-assessment
09:00 – 10:30	Module 1 · R, RStudio, scripts and getting help
10:30 – 10:45	**Tea break**
10:45 – 12:30	Module 2 · Data types, objects and operators
12:30 – 13:30	**Lunch break**
13:30 – 15:00	Vectors, matrices, data frames, conditions and loops
15:00 – 15:15	**Tea break**
15:15 – 16:45	Functions and hands-on practice
16:45 – 17:00	Reflection & wrap-up

DAY 1 · MODULE 1

01

Introduction to R & RStudio

Getting to know the language, the tools and the habits of reproducible work.

In this module you will

- Explain what R, base R, packages and CRAN are
- Navigate the RStudio interface
- Find help with `?` and `??`
- Set the working directory
- Create, save, run and comment an R script

What is R?

- R is a **free, open-source programming language and software environment** for data analysis and statistics.
- Developed at the University of Auckland, New Zealand; first released in 1995.
- Designed as a user-friendly way to do data analysis, statistics and graphics.
- Project homepage: **www.r-project.org**

Base R and packages

- What you install from the R Project website is **base R**. When you need more functionality, you install an **add-on package**.
- Packages are distributed through **CRAN**, the Comprehensive R Archive Network: **cran.r-project.org**

A word of caution

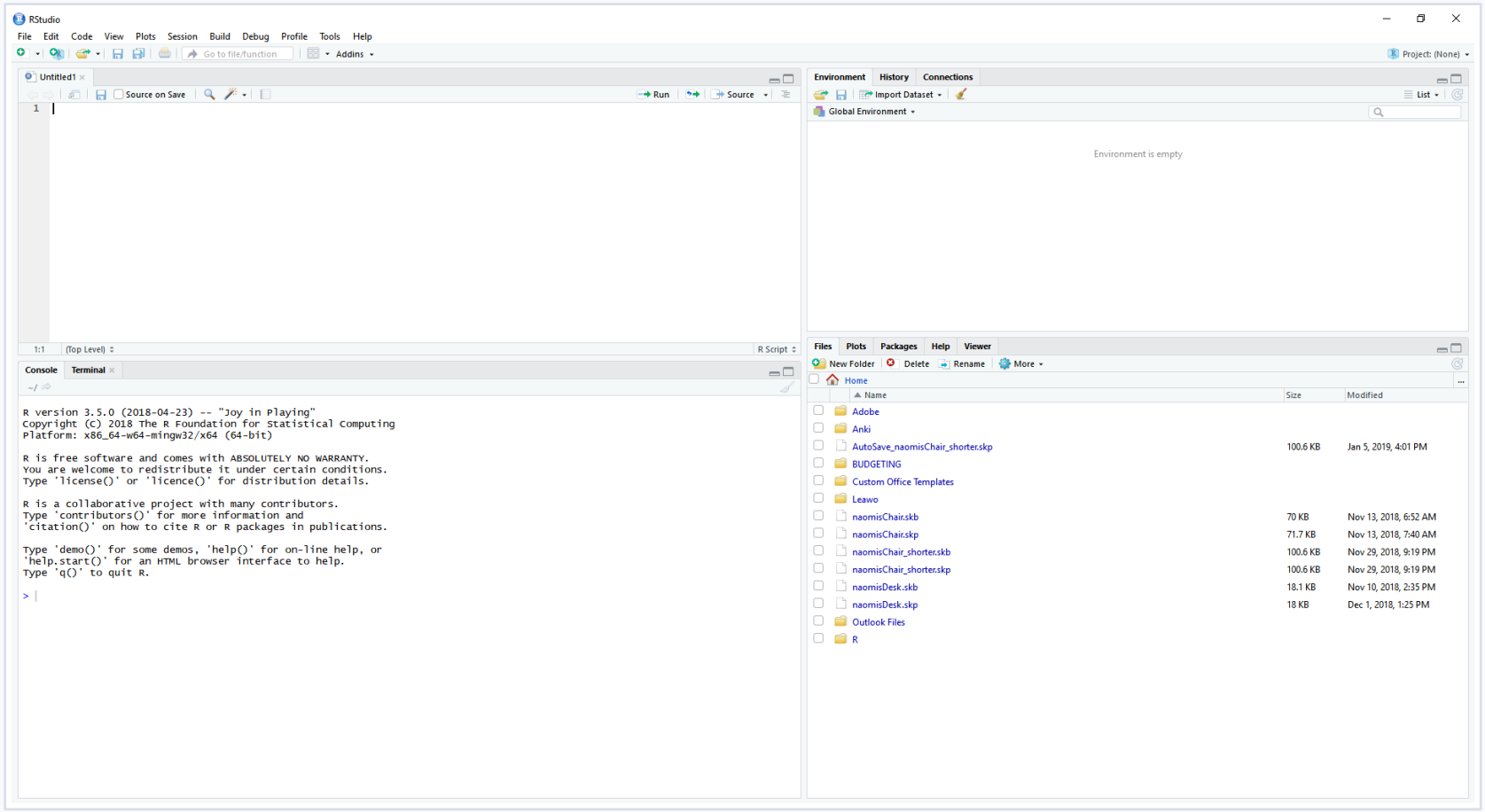
- Anyone can publish a package, so documentation and quality vary.
- Favour packages you know or that are widely recommended in the R community (e.g. the tidyverse).

What is RStudio?

- RStudio is the most widely used **integrated development environment (IDE)** for R. It makes working with R easier through:
  - Syntax highlighting, code completion and smart indentation.
  - A source editor to write R programs and run them directly.
  - Working-directory and file management tools.
  - Integrated help, documentation and graphics viewer.
  - An interactive debugger to diagnose and fix errors quickly.
- Homepage: **posit.co/products/open-source/rstudio**

The RStudio interface

- RStudio is made up of **panes**, each offering a different way to interact with R. Their arrangement can be customised.



The RStudio panes

Console

Type commands interactively and see the output.

Editor

Compose, save and execute scripts.

Environment

View the objects created in the current session.

History

Review the commands you have submitted.

Files

Integrated file explorer.

Plots

Integrated graphics viewer.

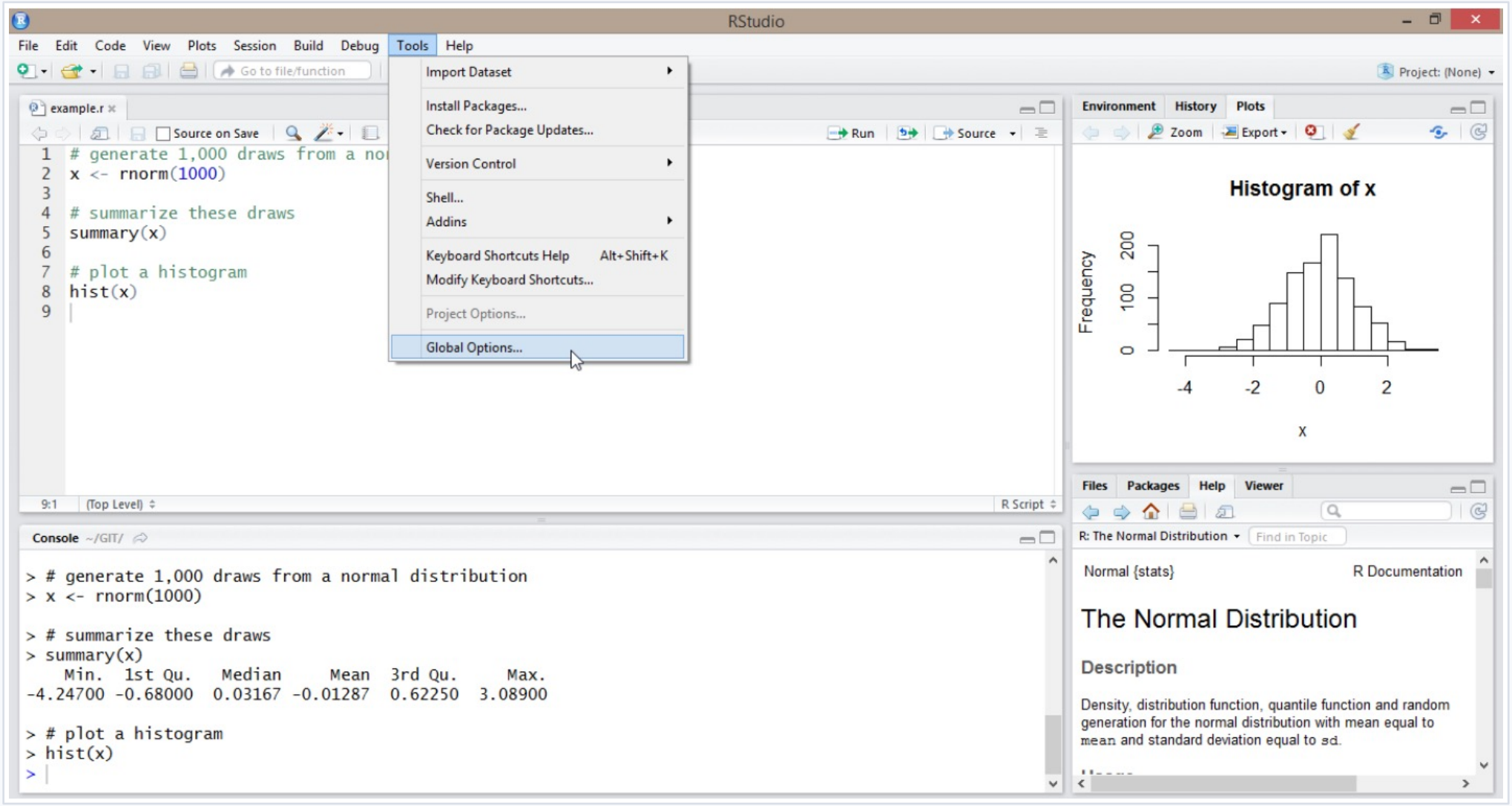
Packages

Installed packages — tick to load.

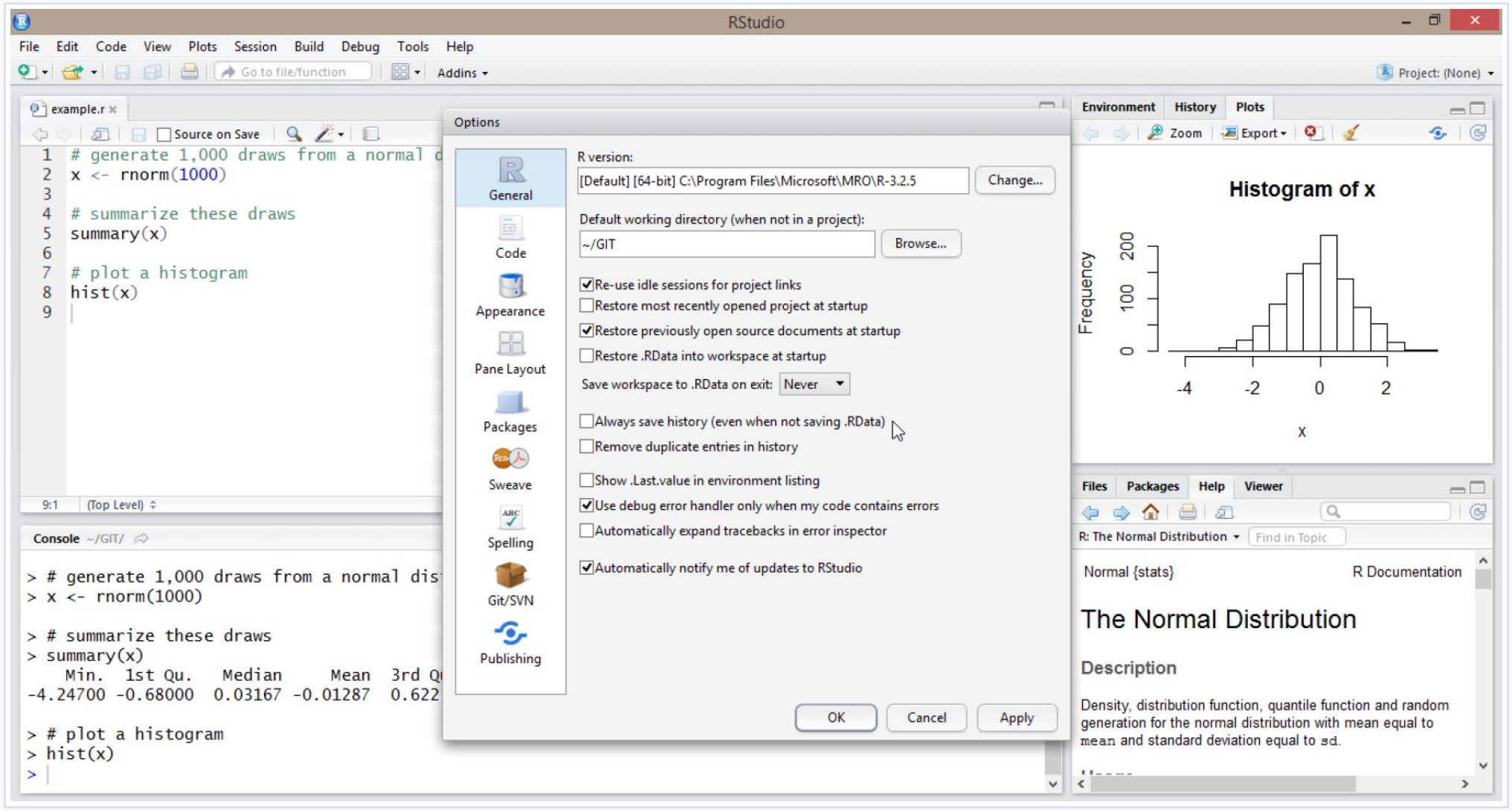
Help

Integrated help-file viewer.

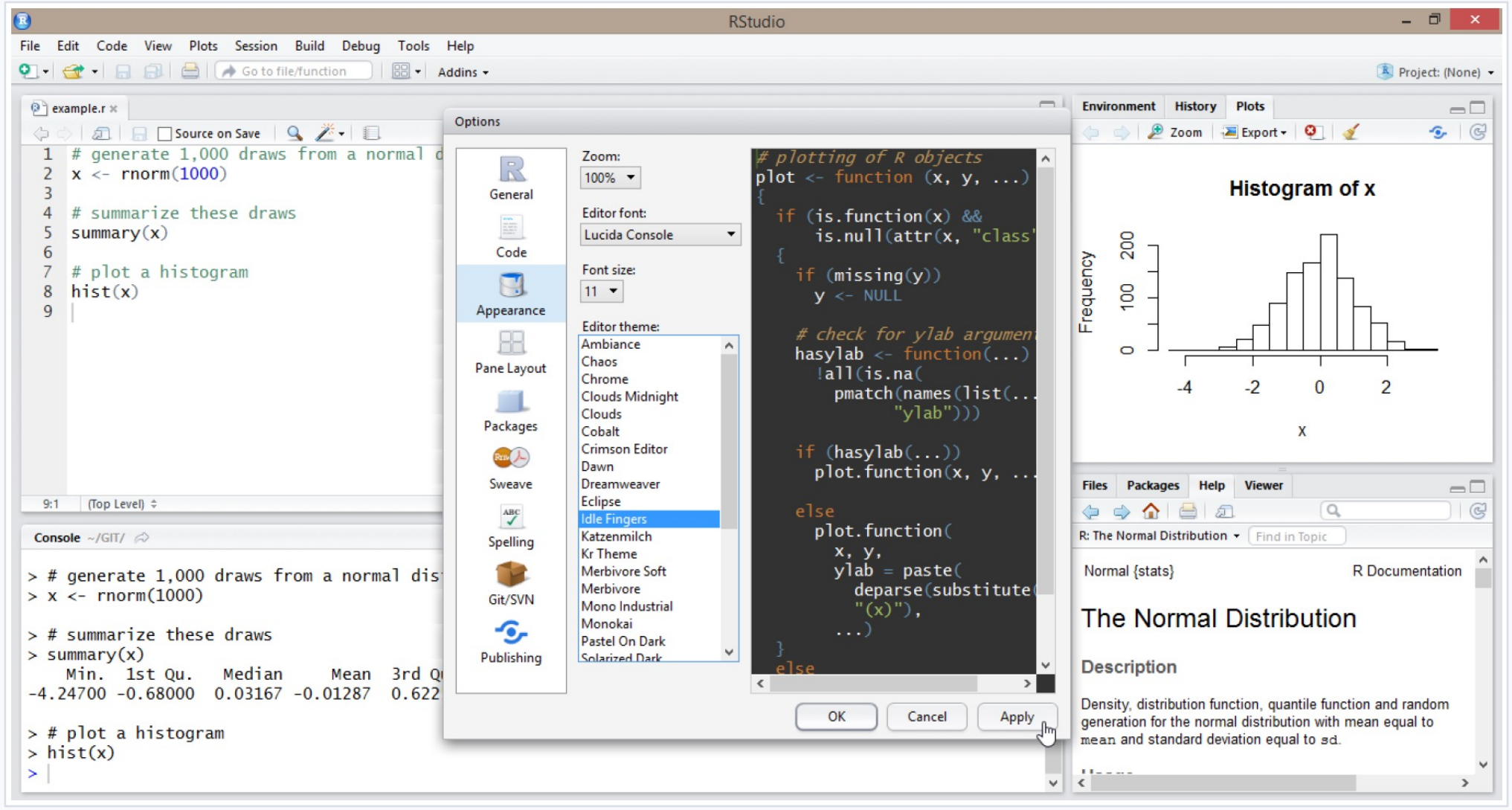
RStudio interface: the console



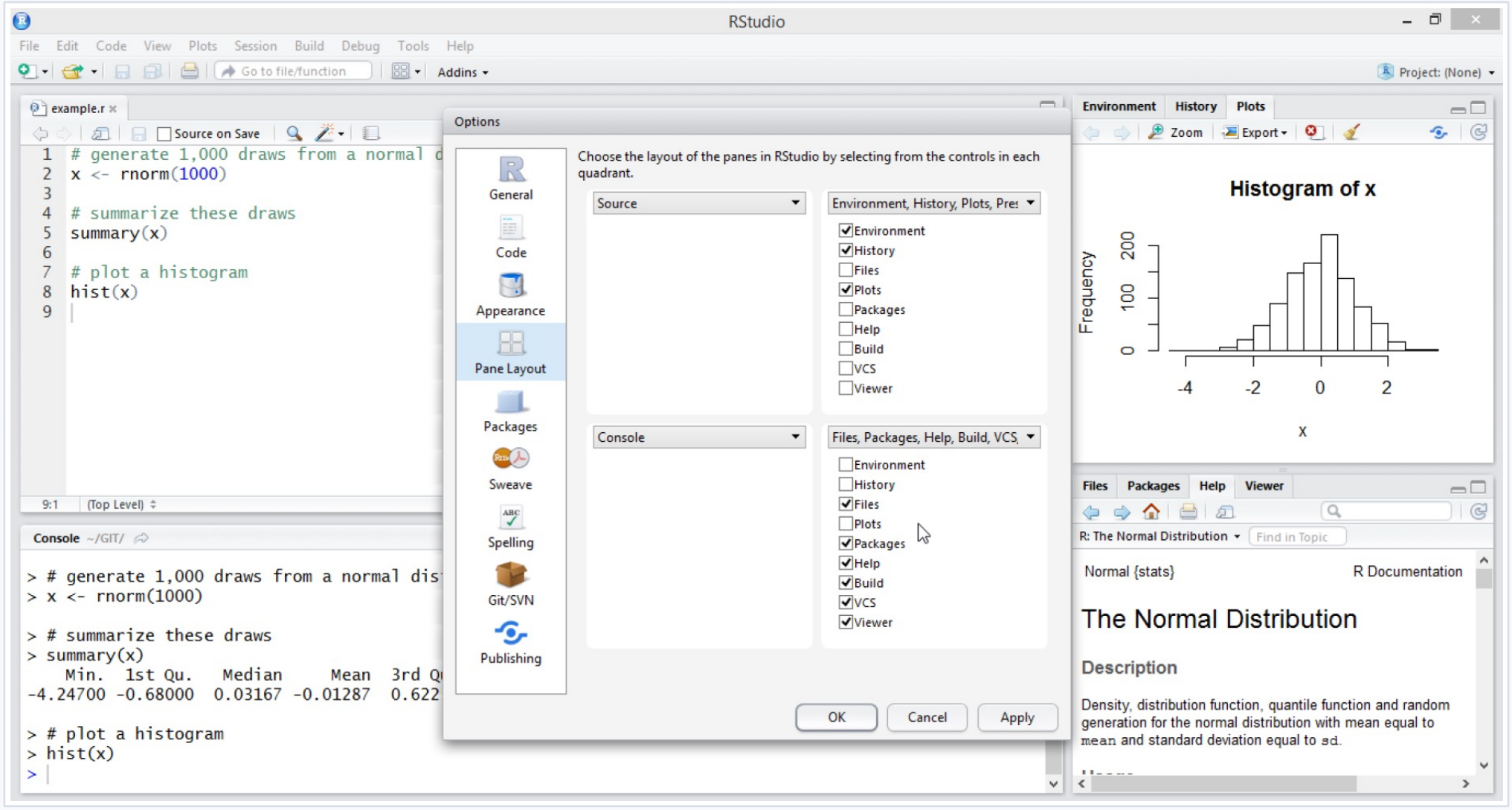
RStudio interface: the source editor



RStudio interface: environment and history



RStudio interface: files, plots, packages and help



Built-in help files: `?function`

Documentation for any R function is one keystroke away: type ?names in the console. Each help page has the same sections.

Description

What the function does.

Usage

Syntax for calling the function.

Arguments

Options that control the function's behaviour.

Details

More detail on how the function behaves.

Value

What the function returns.

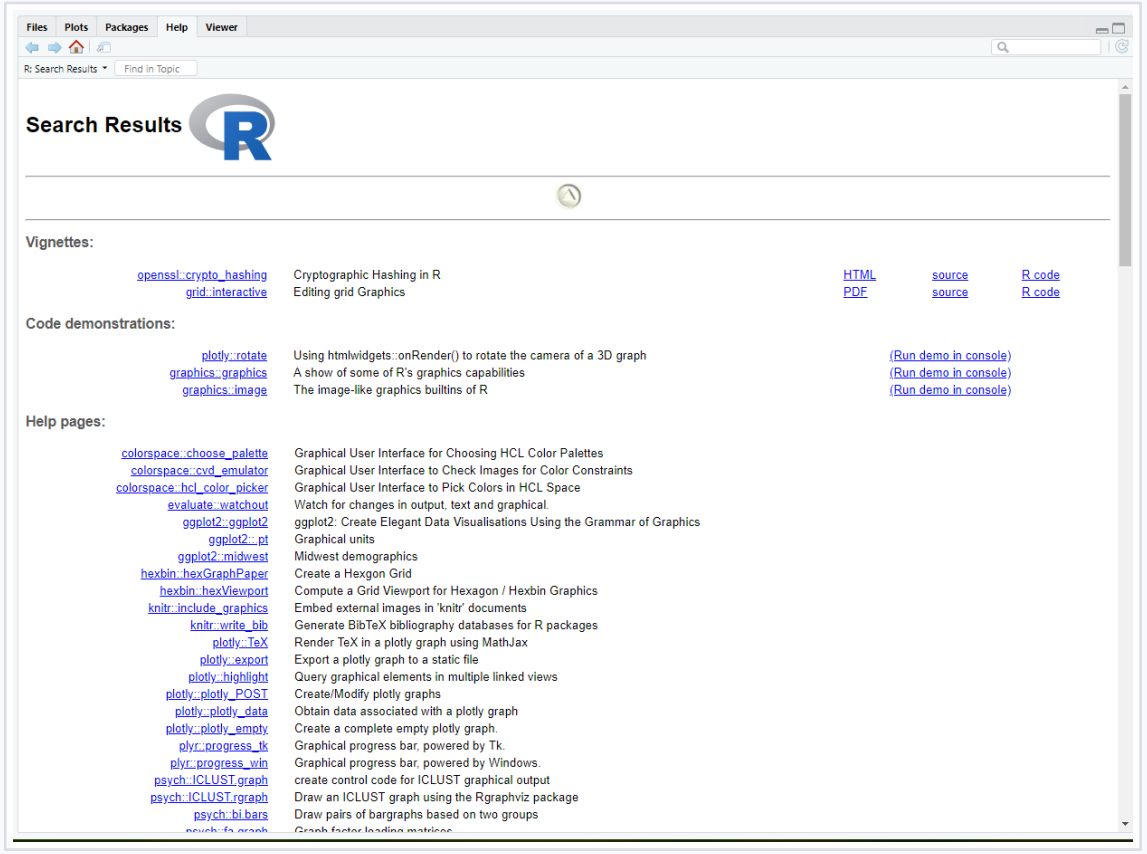
Examples

Working code that uses the function.

Searching the documentation: ??

- When you do not know the function name, search all installed help pages with a double question mark.

```R
??graph
```



CRAN and where else to find help

- **CRAN** is a network of servers worldwide holding identical, up-to-date versions of R code and documentation.
  - Browse **cran.r-project.org**, or search the internet for the package name plus 'CRAN' (e.g. *ggplot2 CRAN*).

More places to look

- **www.r-project.org/help.html** — the official help page.
- **support.posit.co** — RStudio / Posit support articles.
- Stack Overflow (tag `r`) and the package's own website or vignettes.



FACILITATOR TIP

Demonstrate reading one help page live (e.g. ?mean) and show where na.rm is documented — it returns on Day 2.

Setting the working directory

- `setwd()` sets the working directory for the R session — analogous to `cd` in Stata. The path is computer-specific.
- `getwd()` shows the current working directory.

```R
setwd("C:/Rtraining")   # path is computer-specific
getwd()
```

R scripts

- R command files are called **R scripts** (analogous to Stata *do* files). They have the extension `.R` and open in the RStudio editor.
- Scripts are a necessary component of **reproducible research** — and of your sanity.

Opening and saving

- New script: *File* → *New File* → *R Script* or **Ctrl + Shift + N**.
- Save: *File* → *Save* or **Ctrl + S**, or the disk icon in the editor toolbar.

Running a script

- With the script open in RStudio, run the **whole script** with **Ctrl + Shift + Enter**, or a **single line / highlighted block** with **Ctrl + Enter**.
- You can also run a saved script from the console with `source()`:

```
> source(file = "J:/temp/bootcamp_r_training/lectures/lecture_1a_r_basics.r",
+     echo = T)
```

Commenting your code

- R ignores anything on a line after #. Use comments to explain *why* the code does something.

```
> # add 1-5
> 1 + 2 + 3 + 4 + 5
[1] 15
>
> # find the natural log of 10
> log(10)
[1] 2.302585
```



DISCUSSION

What information would a colleague need in the comments to re-run your script next survey round?



BRAINSTORM

10 min

Why should a statistician learn to program?

Before we write more code, let's connect programming to your daily work.

1. Think of a repetitive task you do every survey round in Excel, SPSS or Stata. How long does it take?

2. What goes wrong when the same steps are repeated by hand across many files?

3. What would change if those steps were written once as an R script and re-run each round?

How to run it: Collect answers on a flipchart. Return to this list on Day 3 and tick off what participants can now automate.

TAKE A BREAK

Tea Break

10:30 – 10:45 · 15 minutes

UP NEXT

Module 2 · Data types, objects and operators



DAY 1 · MODULE 2

02

R Fundamentals

The building blocks: data types, objects, operators, data structures and control flow.

In this module you will

- Distinguish numeric, character, logical and factor data
- Check and coerce data types; handle `NA`
- Create objects and name them well
- Use arithmetic, comparison and logical operators
- Work with vectors, matrices and data frames
- Control flow with `if`, `for`, `while` and functions

Basic data types in R

- Computers store data in different ways — as numbers, character strings and other types.
- Pay attention to data types: they determine **which R functions will work** with a given element.
- Access documentation for any type with `?`, e.g. `?numeric`.

The basic data types

numeric

Numbers. Two sub-types: **integer** (whole numbers) and **double** (up to 16 decimal places).

`2254459`, `78.5`

character

Sequences of characters (strings), written in quotes.

`"Oromiya"`

logical

Binary values `TRUE` or `FALSE`.

`as.logical(c(0, 1))`

factor

Categorical data: integer codes with character **levels**.

```
factor(sex, labels =
c("Female", "Male"))
```

Numeric data

- Numbers are stored as **numeric**, of type **integer** (whole numbers, no decimal) or **double** (up to 16 decimal places). They differ in precision.
- Besides missing values, numeric variables can take two special values from arithmetic:
  - `NaN` — *Not a Number*, for impossible values such as `0/0`.
  - `Inf` — infinite values such as `1/0`.

Character data

- Sequences of alphanumeric characters — **strings** — are stored as type **character** and written in quotes.
- Without quotes, R assumes you are referring to an **object** and looks for it.

R

```
print("a")
## [1] "a"

print(a)
## Error in print(a): object 'a' not found
```

Logical data

- Binary (0/1) variables are often stored as type **logical**, taking the values `TRUE` or `FALSE`.
- A numeric 0/1 variable converts to logical with `as.logical()`.

```R
as.logical(c(0, 1, 0, 0))
## [1] FALSE  TRUE FALSE FALSE
```

Factor data

- A **factor** represents a **categorical variable**: integer values with associated character labels called **levels**.
- You work with the labels, not the integers. Factors help R treat text categories as structured levels for summaries and models.
- Convert a numeric or character variable to a factor with `factor()`.

R

```
sex <- c(1, 1, 2)
sex_factor <- factor(sex, levels = c(1, 2), labels = c("Female", "Male"))
print(sex_factor)
## [1] Female Female Male
## Levels: Female Male
```

Factor data: disease severity example

- Levels can be ordered the way you want them to appear in tables and charts.

```R
severity <- c(1, 3, 2, 1, 2)
severity_factor <- factor(severity,
                          levels = c(1, 2, 3),
                          labels = c("Mild", "Moderate", "Severe"))
print(severity_factor)
## [1] Mild     Severe   Moderate Mild     Moderate
## Levels: Mild Moderate Severe
```



DISCUSSION

Which variables in the livestock or crop dataset should be factors? Which should stay numeric?

Determining the data type

- `typeof()` returns the type of a data element.
- The `is.` family of functions tests whether a value has the type you expect.

```R
typeof(5.576)                    ## [1] "double"
typeof("Hello!")                 ## [1] "character"

is.numeric(5)                    ## [1] TRUE
is.integer(5.0)                  ## [1] FALSE
is.character("Isn't coding fun?") ## [1] TRUE
```

Coercion: converting between types

- The `as.` family forces a conversion from one type to another.
- Converting a double to an integer **truncates** the decimals.

```R
years <- c(2024, 2025, 2026)
as.character(years)
## [1] "2024" "2025" "2026"

c <- 78.5
typeof(c)          # "double"
c <- as.integer(c) # coerce to integer
typeof(c)          # "integer"
print(c)           # 78
```

Missing values: NA

- Missing values are stored as `NA` (*Not Available*).
- Test for missing values with `is.na()` — **never** with `== NA`, which returns `NA` itself.

```R
x <- NA
is.na(x)
## [1] TRUE

x == NA
## [1] NA
```

KEY POINT

Any comparison with NA is NA. This is why summary functions return NA when a column has missing values — more on Day 2.

Objects and assignment

- **Objects** are how we store information (data, functions) in R. Each has three components: a **name**, a **value** and **properties** (class, dimension, …).
- Create objects with the **assignment operator** `<-` (or `=`), which assigns contents to a name.

```R
x <- 1
x
## [1] 1
```

Operators in R: arithmetic

1. Arithmetic Operators in R

Operator	Meaning	Example	Result
`+`	Addition	`10 + 5`	`15`
`-`	Subtraction	`10 - 5`	`5`
`*`	Multiplication	`10 * 5`	`50`
`/`	Division	`10 / 5`	`2`
`^`	Exponent / Power	`2 ^ 3`	`8`
`%%`	Modulus (remainder)	`10 %% 3`	`1`
`%/%`	Integer division	`10 %/% 3`	`3`

Operators in R: comparison

2. Comparison Operators in R

Operator	Meaning	Example	Result
`>`	Greater than	`10 > 5`	`TRUE`
`<`	Less than	`10 < 5`	`FALSE`
`>=`	Greater than or equal to	`10 >= 10`	`TRUE`
`<=`	Less than or equal to	`5 <= 10`	`TRUE`
`==`	Equal to	`10 == 10`	`TRUE`
`!=`	Not equal to	`10 != 5`	`TRUE`

> **Note:** In R, `==` tests equality, while `<-` or `=` can be used for assignment. For example: `age <- 30`.

Operators in R: logical connectors

3. Logical Operators / Connectors in R

Operator	Meaning	Example	Result
`&`	AND — both conditions are TRUE	`(10 > 5) & (8 > 3)`	`TRUE`
`|`	OR — at least one is TRUE	`(10 < 5) | (8 > 3)`	`TRUE`
`!`	NOT — reverses TRUE/FALSE	`!(10 < 5)`	`TRUE`
`&&`	AND — checks first element only	`TRUE && FALSE`	`FALSE`
`||`	OR — checks first element only	`TRUE || FALSE`	`TRUE`

Object names

Rules

- Start with a letter (a–z, A–Z).
- Other characters may be letters, digits (0–9), underscores `_` and periods `.`.

Common conventions

- `my_variable_name` — snake_case (recommended)
- `my.variable.name` — period.case
- `myVariableName` — camelCase
- `10_variable_name` — **wrong**: names cannot start with a digit.



QUICK CHECK

Which are valid: `cattle2016`, `2016cattle`, `total.cattle`, `total-cattle`, `Total_Cattle`? Are the last one and `total_cattle` the same object?

TAKE A BREAK

Lunch Break

12:30 – 13:30 · 60 minutes

UP NEXT

Vectors, matrices, data frames, conditions and loops

Data structures in R

- A **data structure** is a way of organising and storing data so it can be accessed, manipulated and analysed efficiently.
- Data structures define how elements are arranged and which operations can be performed on them.
- The common structures in R are the **vector**, **matrix**, **list** and **data frame**.

The four common data structures

Vector

Ordered collection of values of the **same** type.

```
c(1, 2, 3)
```

Matrix

Two-dimensional vector — rows × columns, one type.

```
matrix(1:6, nrow = 2)
```

List

Ordered collection of objects of **any** type, even other lists.

```
list(1, "a", TRUE)
```

Data frame

Spreadsheet-like table: columns are variables (one type each), rows are observations.

```
data.frame(...)
```

Vectors: creating with `c()`

- A vector is an ordered collection of values of the same basic type. Create one manually with the **combine** function `c()`.

```
> iso3 <- c("CAN", "USA", "MEX")
> iso3
[1] "CAN" "USA" "MEX"

> pop <- c(35.16, 318.9, 122.3)
> pop
[1]  35.16 318.90 122.30

> admin1 <- c(13L, 51L, 31L)
> admin1
[1] 13 51 31

> spanish <- c(FALSE, FALSE, TRUE)
> spanish
[1] FALSE FALSE  TRUE
```

Vectors: sequences and repetition

- Numeric sequences with `seq()` or the colon operator `:`; repeat elements with `rep()`.

```
> seq(from = 0, to = 27, by = 3)
 [1]  0  3  6  9 12 15 18 21 24 27
> 1990:2000
 [1] 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000
```

```
> rep(c(1, 2, 3), each = 2)
[1] 1 1 2 2 3 3
> rep(c("red", "blue", "green"), times = 2)
[1] "red"   "blue"  "green" "red"   "blue"  "green"
```

Matrices

- A matrix is a **two-dimensional vector** of values of the same type, usually created by assigning dimensions to a vector.

```
> mat <- matrix(1:6, nrow = 2, ncol = 3, byrow = T)
> mat
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

Matrices: teff yield of two regions

- Example: representing the teff yield of two regions as a matrix.

```
Yield<-matrix(c(500,600,700,
                450,550,650), nrow = 2, byrow = TRUE)
print(Yield)
rownames(Yield)<-c("Oromia","Amhara")
colnames(Yield)<-c("Teff","Maize","Wheat")
print(Yield)
```

Matrices: row and column names

- Assign `rownames()` and `colnames()` to make the matrix self-explanatory.

```
> print(Yield)
     [,1] [,2] [,3]
[1,]  500  600  700
[2,]  450  550  650
> rownames(Yield)<-c("Oromia","Amhara")
> colnames(Yield)<-c("Teff","Maize","Wheat")
> print(Yield)
       Teff Maize Wheat
Oromia  500   600   700
Amhara  450   550   650
```

Matrices: summaries by row and column

- `rowSums()`, `colSums()`, `rowMeans()` and `colMeans()` summarise across a dimension.

```
#per region yield
yieldpercrop<-rowSums(Yield)
print(yieldpercrop)

# per crop yiled
totalyield<-colSums(Yield)
print(totalyield)
```

```
> #per region yield
> yieldpercrop<-rowSums(Yield)
> print(yieldpercrop)
Oromia Amhara
  1800   1650
> # per crop yiled
> totalyield<-colSums(Yield)
> print(totalyield)
 Teff Maize Wheat
  950  1150  1350
```

Data frames

- A **data frame** stores data in spreadsheet-like form — analogous to a dataset in Stata.
- Columns are **variables** (each of one data type); rows are **observations** (which can mix types across columns).

```R
class(livestock)
## [1] "data.frame"
```

Data frames: constructing from vectors

- Combine vectors of equal length into a data frame with `data.frame()`.

```
> iso3 <- c("CAN", "USA", "MEX")
> pop <- c(35.16, 318.9, 122.3)
> admin1 <- c(13L, 51L, 31L)
> spanish <- c(FALSE, FALSE, TRUE)
```

```
> df <- data.frame(iso3, pop, admin1, spanish)
> df
  iso3    pop admin1 spanish
1  CAN  35.16     13   FALSE
2  USA 318.90     51   FALSE
3  MEX 122.30     31    TRUE
```

Conditional statements

- Conditional statements let a program **make decisions**: different blocks of code run depending on whether a condition is `TRUE` or `FALSE`.
- In R they are written with `if`, `else if` and `else`.

R

```
if (condition) {
  true_action
}

if (condition) {
  true_action
} else {
  false_action
}
```

Conditional statements: `if` and `else`

- **1 · `if`** checks a single condition.
- **2 · `else`** runs an alternative when the first condition is false.

```r
x <- 10
if (x > 5) {
  print("x is greater than 5")
}
```

```r
yiled<-85
if(yiled>=90){
  print( "The yield is good")
}else {
  print("The yiled is not as expected")
}
```

Conditional statements: `if … else if … else`

- **3** · Test several conditions in order; the first TRUE branch runs.

```
#if...else...if
yiled<-85
if(yiled>=90){
  print( "The yield is excellent ")
}else if(yiled>=70){
  print("The yiled is moderated")
}else{
  print("the yield is not as expected")
}
```

Loops: the `for` loop

- Loops **repeatedly execute** a block of code for a defined number of iterations or until a condition is met. R offers `for` and `while`.

```r
for (variable in sequence) {
  # code to be repeated
}

for (i in 1:5) {
  print(i)
}
# Output: 1 2 3 4 5
```

Loops: iterating over crops

- Example 2 — iterate through three crops and print each name.

```
crops <- c("Teff","Wheat","Maize")
for (cr in  crops){
  print(paste(" I like ", cr))  #
}
```

```
[1] " I like  Teff"
[1] " I like  Wheat"
[1] " I like  Maize"
```

Loops: total yield of five Oromia zones

- Example 3 — accumulate a total inside the loop.

```
Oromia_yield<-c(20,40,30,70,20)
total<-0
for( yld in Oromia_yield){
  total<-total + yld
}
print(paste0("total yiled of teff in five zones of oroimia ",total))
```

```
[1] "total yiled of teff in five zones of oroimia 180"
```

Loops: average yield of five Amhara zones

- Example 4 — average yield (tonnes). Output: **134.6**

```
yield<- c(120, 135, 140, 128, 150)
sum_yld <- 0
for (i in yield) {
  sum_yld <- sum_yld + i
}
average_yld <- sum_yld / length(yield)
print(average_yld )
```



DISCUSSION

Could this loop be replaced by a single function call? Which one — and when is a loop still the right tool?



HANDS-ON EXERCISE

25 min

Practice: types, decisions and repetition

1. Create a vector of five zone names and a vector of their cattle counts. Check the type of each with `typeof()`.

2. Convert a 1/2/3 geography code vector into a factor with the labels Lowland, Midland and Highland.

3. Write an `if … else if … else` that prints 'low', 'medium' or 'high' for a cattle count stored in an object.

4. Use a `for` loop over `2014:2018` to print each survey year; then compute the total of your cattle vector with a loop and compare with `sum()`.

How to run it: Pairs. After 20 minutes two pairs show their solutions; highlight vectorised sum() versus the loop.

TAKE A BREAK

Tea Break

15:00 – 15:15 · 15 minutes

UP NEXT

Functions and hands-on practice



Functions

- A **function** is a reusable block of code that performs a specific task when called.
- It takes inputs (**arguments**), processes them and **returns** an output.

```R
function_name <- function(arguments) {
  # code to be executed
  return(result)
}
```

Functions: total livestock

- Example 1 — a function that returns the sum of livestock: **90000**.

```r
#  A simple function that computes total livestock for one zone (sum of species)

total_livestock <- function(cattle, sheep, goats) {
  cattle + sheep + goats
}

#Use or call the function to compute the values based on the rule
print(total_livestock(2000, 3000, 4000))
```

Functions: categorising zones by cattle count

- Example 2 — a function that returns a category label for a cattle count.

```r
# classify a single zone into category large meddium and small based on thier cattle size
classify_zone <- function(cattle_count) {
  if (cattle_count > 800000) {
    "Large"
  } else if (cattle_count > 200000) {
    "Medium"
  } else {
    "Small"
  }
}

# classify cattle count  based on  the function rule
classify_zone(50000)
```



DISCUSSION

What repetitive calculation in your work could become a function? Name it and list its arguments.



HANDS-ON EXERCISE

35 min

MODULE 2 · R FUNDAMENTALS

Day 1 hands-on: putting it together

1. Build a data frame with three regions, their cattle totals for 2017 and 2018.

2. Write a function `growth(old, new)` that returns the percentage change between two values, and apply it to each region with a loop.

3. Write a function that returns 'above' or 'below' depending on whether a value exceeds the mean of a vector.

4. Stretch: add a `Geography` factor column to your data frame and print `summary()` of it.

How to run it: Circulate and help with syntax errors. Emphasise reading the error message — it usually says exactly what is wrong.



DISCUSSION

15 min

Day 1 reflection

Take five minutes to write, then we go around the room.

1. One thing I learned today that I can use immediately.
2. One thing that is still unclear and I would like revisited tomorrow.
3. One question I want to be able to answer with R by the end of the training.

How to run it: Note the 'unclear' items on the flipchart and address them in the Day 2 recap.

TRAINING DAY

2

Saturday, 19 September 2026

Importing, exploring and preparing data — in base R and the tidyverse

Today's schedule

08:30 – 09:00	Recap of Day 1 & questions
09:00 – 10:30	Module 3 · Packages, importing data and first look at a data frame
10:30 – 10:45	**Tea break**
10:45 – 12:30	Module 4 · Summaries, frequency tables, indexing and missing values
12:30 – 13:30	**Lunch break**
13:30 – 15:00	Subsetting, new columns, sorting · Module 5 · dplyr verbs
15:00 – 15:15	**Tea break**
15:15 – 16:45	Grouped summaries, appending and merging datasets
16:45 – 17:00	Reflection & wrap-up



DISCUSSION

15 min

Recap of Day 1

1. In one sentence: what is the difference between a vector and a data frame?

2. Why does `x == NA` not tell you whether `x` is missing? What should you use instead?

3. Show your neighbour the function you wrote yesterday. What are its arguments and what does it return?

How to run it: Address the 'still unclear' items collected yesterday before moving on.

DAY 2 · MODULE 3

03

Packages & Importing Data

Extending R with packages and bringing ESS datasets into a data frame.

In this module you will

- Install, load and get help on a package
- Explain what the tidyverse is
- Clear the workspace and set the working directory
- Read CSV, Excel, Stata and SPSS files
- View a data frame and inspect its structure

What is an R package?

- An **R package** is a collection of functions, datasets and documentation bundled together to extend R for specific tasks — e.g. `ggplot2` for graphics, `dplyr` for data manipulation.

Four steps to use a package

- **Install** it once: `install.packages("package_name")`
- **Load** it in every new session: `library(package_name)`
- **Use** its functions, e.g. `filter()` from dplyr.
- **Get help**: `?function_name` or `help(package = "package_name")`

Installing and loading a package

- Install **once** — from the RStudio *Packages* pane or with code. Installed packages stay until you remove them.
- Load the package **each time** you open a new R session before using its functions.
- The *Packages* pane lists everything installed on your machine.

R

```
install.packages("tidyverse")   # once
library(tidyverse)              # every session
```

The tidyverse

- The **tidyverse** is a collection of R packages designed to make data analysis, cleaning and visualisation **easier, faster and more consistent**.

Main uses

- Import data · clean and transform · analyse and summarise · visualise results.

Key packages inside the tidyverse

- `ggplot2` — visualisation · `dplyr` — filter, select, summarise · `tidyr` — reshape messy data
- `readr` — read CSV and text files · `tibble` — an improved data frame · `stringr` — work with text

Clearing the workspace

- Sometimes you want to start clean — old objects can get confused with new definitions.
- `rm(list = ls())` removes all user-created objects and functions from the session.

```R
rm(list = ls())
```


KEY POINT

Put rm(list = ls()) and setwd() at the top of every script so it always starts from a known state.

Datasets for this training

- We mainly use the **2014–2018 livestock dataset**: cattle, sheep, goats, camels and other species for Ethiopian regions and zones, with inflow / outflow / net **Type**.
- It also contains **dairy data** — dairy production, dairy animals and related information.
- The **crop dataset** for 2014 is used for the exercises.

Reading a CSV file

- `read.csv()` loads tabular data into a data frame.
- By default text columns may become factors; set `stringsAsFactors = FALSE` to keep them as characters.

```R
livestock <- read.csv("livestock.csv")

# full path, and keep text as character
livestock <- read.csv("C:/Rtraining/livestock.csv", stringsAsFactors = FALSE)
```

Reading other file formats

- **Excel** — package `readxl`; **Stata** and **SPSS** — package `haven`.

```R
install.packages("readxl")          # run once
library(readxl)
data_excel <- read_excel("path/to/file.xlsx")

install.packages("haven")           # run once
library(haven)
data_stata <- read_dta("path/to/well_being.dta")
data_spss  <- read_sav("Breastcancer.sav")
```

File paths

- Use **forward slashes**: `"C:/path/to/dir/"` is right; `"C:\path\to\dir\"` is wrong.
- Paths can be **absolute** or **relative** to the working directory.

```r
# absolute
data <- read.csv("C:/files/training/data.csv")

# relative to the working directory
setwd("C:/files/training")
data <- read.csv("data.csv")
```

Reading in data: tips

Header row

Does the first row hold the variable names? If not, use header = FALSE.

Variable names

Files from other programs may have names R dislikes (spaces from Excel). Rename them.

Garbage rows

Comments or notes above the data? Use skip = n to reach the actual table.

Missing codes

Know how missing values are coded (e.g. -999, '.') and convert them to NA with na.strings.

Let's load the livestock data

- Start clean, set the working directory, read the file and confirm the object class.

```R
rm(list = ls())
setwd("C:/Rtraining/resources")
livestock <- read.csv("livestock.csv")

class(livestock)
## [1] "data.frame"
```

Viewing the data: `View()`

- Like Stata's browser, RStudio has a data viewer: `View(livestock)` opens the table in a tab.

Filter

	Level	Region	Zone_Name	CATTLE	SHEEP	GOATS	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES
1	National	National	National	66260987	38013272	45716091	2138928	357743	10020518	6978053	41350684	5982336
2	Region	Afar	Afar	2085194	3625830	7627510	98	91	276257	1003978	63227	18689
3	Zone	Afar	Zone_1 (Awsiresu)	930591	1377823	3251744	NA	NA	103573	325483	NA	NA
4	Zone	Afar	Zone_2 (Kelbetiresu)	62214	413083	1376619	NA	NA	59030	123434	12821	18689
5	Zone	Afar	Zone_3 (Gebiresu)	733077	1112492	1328252	NA	91	50378	324045	37825	NA
6	Zone	Afar	Zone_4 (Fentiresu)	100838	338140	587919	98	NA	24433	66140	NA	NA
7	Zone	Afar	Zone_5 (Hariresu)	258475	384293	1082975	NA	NA	38842	164875	12580	NA
8	Region	Amhara	Amhara	17293813	10267358	6808301	518513	218858	3879122	39184	16334631	1330300
9	Zone	Amhara	North Gonder	761124	507316	557070	68943	13642	209833	NA	1062025	168138
10	Zone	Amhara	South Gonder	1972622	1162805	662987	39516	30363	425555	NA	1778314	121087
11	Zone	Amhara	North Wello	924892	928259	583226	37547	16326	293698	14847	1069976	48456
12	Zone	Amhara	South Wello	1952374	1538406	1390763	40250	30134	664223	2642	1965306	156088
13	Zone	Amhara	North Shewa	1690086	1770234	844706	97838	10081	606883	11242	1660325	67901
14	Zone	Amhara	East Gojjam	2185995	1225349	315053	79534	18714	473619	NA	1087572	126520
15	Zone	Amhara	West Gojjam	3003155	1653139	303838	31268	70698	446980	NA	3282574	208195
16	Zone	Amhara	Waghimra	367985	196608	550710	NA	2645	122909	NA	398167	135226

Viewing the data: `head()` and `tail()`

- `head()` prints the first 6 rows by default; the argument `n` sets the number. `tail()` prints the last *n* rows.

```
> head(livestock)
     Level   Region            Zone_Name    CATTLE    SHEEP    GOATS  HORSES  MULES  DONKEYS  CAMELS  POULTRY BEEHIVES
1 National National             National 66260987 38013272 45716091 2138928 357743 10020518 6978053 41350684  5982336
2   Region     Afar                 Afar  2085194  3625830  7627510      98     91   276257 1003978    63227    18689
3     Zone     Afar    Zone_1 (Awsiresu)   930591  1377823  3251744      NA     NA   103573  325483       NA       NA
4     Zone     Afar Zone_2 (Kelbetiresu)    62214   413083  1376619      NA     NA    59030  123434    12821    18689
5     Zone     Afar    Zone_3 (Gebiresu)   733077  1112492  1328252      NA     91    50378  324045    37825       NA
```

```
> tail(livestock)
   Level    Region     Zone_Name CATTLE SHEEP  GOATS HORSES MULES DONKEYS CAMELS POULTRY BEEHIVES
82  Zone   Gambela       Angewak   7636   284  10196     NA     NA      381     NA   44592    11537
83  Zone   Gambela         Newer 267613 17740  83614     NA     NA       NA     NA  173430       NA
84  Zone   Gambela      Mejenger  20171 14270   8466    760    727      234     NA   65328    33804
85  Zone   Gambela Itang Special  30542  1342  14739     NA     NA       NA     NA   31520       NA
86  Zone    Harari        Hareri  74902  4570 105248     NA     NA    11369    821   85180     4372
```

Data frame information

- Useful functions describe the data frame's size, names and structure.

```r
dim(livestock)    # number of rows & columns
nrow(livestock)   # number of rows
ncol(livestock)   # number of columns
names(livestock)  # column names
```

```
> names(livestock)
 [1] "Level"     "Region"    "Zone_Name" "CATTLE"    "SHEEP"     "GOATS"     "HORSES"    "MULES"     "DONKEYS"   "CAMELS"    "POULTRY"
[12] "BEEHIVES"
```

Data frame structure: `str()`

- `str()` summarises the object's structure — class, dimensions, variable names, data types and a preview of each column.

```
> str(livestock)
'data.frame':   87 obs. of  12 variables:
 $ Level     : chr  "National" "Region" "Zone" "Zone" ...
 $ Region    : chr  "National" "Afar" "Afar" "Afar" ...
 $ Zone_Name: chr  "National" "Afar" "Zone_1 (Awsiresu)" "Zone_2 (Kelbetiresu)" ...
 $ CATTLE    : num  66260987 2085194 930591 62214 733077 ...
 $ SHEEP     : num  38013272 3625830 1377823 413083 1112492 ...
 $ GOATS     : num  45716091 7627510 3251744 1376619 1328252 ...
 $ HORSES    : num  2138928 98 NA NA NA ...
 $ MULES     : num  357743 91 NA NA 91 ...
 $ DONKEYS   : num  10020518 276257 103573 59030 50378 ...
 $ CAMELS    : num  6978053 1003978 325483 123434 324045 ...
 $ POULTRY   : num  41350684 63227 NA 12821 37825 ...
 $ BEEHIVES : num  5982336 18689 NA 18689 NA ...
```



HANDS-ON EXERCISE

15 min

Exercise: first look at the crop dataset

1. Read the crop dataset into an object called `crop`.
2. Use `head()` to print the first 10 rows and `tail()` for the last 10.
3. How many rows and columns does it have? List the variable names and check the structure with `str()`.

TAKE A BREAK

Tea Break

10:30 – 10:45 · 15 minutes

UP NEXT

Module 4 · Summaries, frequency tables, indexing and missing values



DAY 2 · MODULE 4

04

Exploring & Preparing Data in Base R

Summary statistics, frequency tables, indexing, missing values, subsetting and new columns.

In this module you will

- Compute summary statistics and handle `NA` with `na.rm`
- Build frequency and proportion tables
- Index rows and columns of a data frame
- Find, fill and drop missing values
- Subset rows with logical conditions
- Create, recode, drop and sort columns

Summary statistics

- R has many functions for summary statistics: `mean()`, `sd()`, `quantile()`, `var()`, `median()`, `min()`, `max()`, `range()`, `sum()`, `table()`.
- Use `$` to pick one column of a data frame.

```R
mean(livestock$CATTLE)
## [1] 2254459
sd(livestock$CATTLE)
## [1] 7791702
quantile(livestock$CATTLE, c(0, 0.1, 0.25, 0.5, 0.75, 0.9, 1))
##     0%     10%     25%     50%     75%     90%     100%
## 7636.0 72709.2 318480.5 688007.0 1272388.5 2544339.6 66260987.0
```

Summary statistics with missing values

- If a column contains NA, most summary functions return NA — or an error.

```r
mean(livestock$HORSES)
## [1] NA
sd(livestock$HORSES)
## [1] NA
quantile(livestock$HORSES, c(0, 0.25, 0.5, 0.75, 1))
## Error in quantile.default(...):
##   missing values and NaN's not allowed if 'na.rm' is FALSE
```

Overriding the default: `na.rm = TRUE`

- The default of most R functions is to **retain** missing values. Override it with `na.rm = TRUE` to get sensible results.

```r
mean(livestock$HORSES, na.rm = TRUE)      # mean of horse population
sd(livestock$HORSES, na.rm = TRUE)        # standard deviation
quantile(livestock$HORSES, c(0, 0.1, 0.25, 0.5, 0.75, 0.9, 1), na.rm = TRUE)
```

```
> quantile(livestock$HORSES, c(0, 0.1, 0.25, 0.5, 0.75, 0.9, 1), na.rm = TRUE)
      0%      10%      25%      50%      75%      90%     100%
   98.00   416.00  4820.00 25876.00 69194.25 216681.00 2138928.00
```

HANDS-ON EXERCISE

Find the mean, standard deviation and quartiles of the SHEEP, GOATS and CAMELS populations.

summary(): the whole data frame at once

- summary() reports the number of NAs, basic statistics for numeric variables and the frequency of each level for factors and logicals.

```r
summary(livestock)
```

```
> summary(livestock)
   Level              Region            Zone_Name            CATTLE              SHEEP               GOATS              HORSES
 Length:87          Length:87          Length:87          Min.   :    7636   Min.   :     284   Min.   :    8466   Min.   :     98
 Class :character   Class :character   Class :character   1st Qu.:  318480   1st Qu.:   91548   1st Qu.:  111132   1st Qu.:   4820
 Mode  :character   Mode  :character   Mode  :character   Median :  688007   Median :  332091   Median :  306316   Median :  25876
                                                          Mean   : 2254459   Mean   : 1319938   Mean   : 1567030   Mean   : 114200
                                                          3rd Qu.: 1272388   3rd Qu.:  850221   3rd Qu.:  845768   3rd Qu.:  69194
                                                          Max.   :66260987   Max.   :38013272   Max.   :45716091   Max.   :2138928
                                                                             NA's   :1                             NA's   :31
     MULES            DONKEYS            CAMELS            POULTRY            BEEHIVES
 Min.   :    37   Min.   :     234   Min.   :    222   Min.   :     183   Min.   :    165
 1st Qu.:  1048   1st Qu.:   23754   1st Qu.:  11431   1st Qu.:  102034   1st Qu.:  20480
 Median :  2776   Median :   79962   Median :  81256   Median :  358730   Median :  57177
 Mean   : 16746   Mean   :  352237   Mean   : 565534   Mean   : 1493924   Mean   : 238249
 3rd Qu.:  8386   3rd Qu.:  222822   3rd Qu.: 324045   3rd Qu.:  969820   3rd Qu.: 130873
 Max.   :357743   Max.   :10020518   Max.   :6978053   Max.   :41350684   Max.   :5982336
 NA's   :23       NA's   :2          NA's   :50        NA's   :5          NA's   :12
>
```

`summary()` for specific columns


```
> summary(livestock$CATTLE)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   7636  318480  688007 2254459 1272388 66260987
```

```
> quantile(livestock$CATTLE, c(0.25, 0.5, 0.75), na.rm = TRUE)
     25%      50%      75%
318480.5 688007.0 1272388.5
```



HANDS-ON EXERCISE

Summarise the goat and sheep populations.

Unique values: `unique()`

- `unique()` displays each distinct value of a column once — check the spelling of Region names here.

```
> unique(livestock$Region) # tells the unique values in the Region column
 [1] "Afar"          "Amhara"        "Oromiya"       "Oromia"        "Somalia"       "Benishangul"   "SNNP"          "Sidama_total"
 [9] "Gambela"       "Harari"        "Dire Dawa"
```

```
> unique(livestock$Zone) #tells the unique values  within  the zone column
 [1] "Zone_1 (Awsiresu)"     "Zone_2 (Kelbetiresu)"  "Zone_3 (Gebiresu)"     "Zone_4 (Fentiresu)"    "Zone_5 (Hariresu)"
 [6] "North Gonder"          "South Gonder"          "North Wello"           "South Wello"           "North Shewa"
[11] "East Gojjam"           "West Gojjam"           "Waghimra"              "Awi"                   "Oromiya Special Zone"
[16] "Central Gonder"        "West Gonder"           "West Wollega"          "East Wollega"          "Ilu Ababor"
[21] "Jimma"                 "West Shewa"            "East Shewa"            "Arsi"                  "West Hararghe"
[26] "East Hararge"          "Bale"                  "Borena"                "South West Shewa"      "Guji"
[31] "West Guji"             "West Arsi"             "Kelem Wollega"         "Horo Guduru Wollega"   "Buno Bedele"
[36] "East Bale"             "Siti"                  "Fafan"                 "Jerer"                 "Erer"
[41] "Korahe"                "Shebele"               "Dollo"                 "Afder"                 "Liben"
[46] "Nogob"                 "Dawa"                  "Assosa"                "Metekel"               "Kemashi"
[51] "Mao Ena Komo"          "Gurage"                "Hadiya"                "Kembata Tembaro"       "Gedeo"
[56] "Welayta"               "South Omo"             "Sheka"                 "Kefa"                  "Gamo"
[61] "Bench Maji"            "Yem Liyu"              "Segen Akababi Hizboch" "Alaba"                 "Dawro"
[66] "Basketo Special"       "Konta Liyu"            "Silite"                "Gofa"                  "Konso"
[71] "Sidama"                "Angewak"               "Newer"                 "Mejenger"              "Itang Special"
[76] "Hareri"                "Dire Dawa"
```

Frequency tables: `table()`

- `table()` builds one-way or two-way frequency tables.

```
> table(livestock$Type, livestock$Region) # Gives the frequence of livestock inflow,outflow and Net for each region

         Afar Amhara Benishangul Dire Dawa Gambela Harari Oromia Oromiya Sidama_total SNNP Somalia
  inflow    3      4           2         0       1      1       0       5             0     7       6
  Net       1      2           0         1       1      0       1       7             0     5       2
  outflow   1      6           2         0       2      0       1       7             1     7       3
```

Frequency tables: labelled dimensions

- `deparse.level = 2` uses the full expressions you supplied as dimension names.

```
> table(livestock$Type, livestock$Region,deparse.level = 2)
              livestock$Region
livestock$Type Afar Amhara Benishangul Dire Dawa Gambela Harari Oromia Oromiya Sidama_total SNNP Somalia
       inflow     3      4           2         0       1      1       0       5             0     7       6
       Net        1      2           0         1       1      0       1       7             0     5       2
       outflow    1      6           2         0       2      0       1       7             1     7       3
>
```

Frequency tables: dealing with missing values

- `useNA = 'ifany'` — include an `NA` count if there are any.
- `useNA = 'always'` — always include an `NA` row, even if the count is zero.
- `useNA = 'no'` — never include `NA` (the default).

```
# Missing value handling
table(livestock$Type, livestock$Region,  useNA = 'ifany',deparse.level = 2)#includes NA in table if there any
table(livestock$Type, livestock$Region,  useNA = 'always',deparse.level = 2) #includes NA row even if NA is not there
table(livestock$Type, livestock$Region,  useNA = 'no',deparse.level = 2) # does not include NA in the table
```

Frequency tables: proportions

- `prop.table()` converts counts into proportions; multiply by 100 for percentages.

```
TableA<-table(livestock$Type, livestock$Region,  useNA = 'ifany',deparse.level = 2)
TableA
prop.table(TableA) # conversts the frequency  table in to proportion

#percentage of prop table
prop.table(TableA) *100


# Row percentage of the  prop table is equivalent to 3/79 for afar inflow where 79 is the totc
prop.table(TableA, margin = 1) *100

#column percentage of the prop table
prop.table(TableA, margin = 2) *100
view(prop.table(TableA, margin = 2) *100)
```

Frequency tables: proportion and percentage


```
> prop.table(TableA) # conversts the frequency  table in to proportion
              livestock$Region
livestock$Type       Afar     Amhara Benishangul  Dire Dawa    Gambela     Harari     Oromia    Oromiya Sidama_total       SNNP
       inflow  0.03797468 0.05063291  0.02531646 0.00000000 0.01265823 0.01265823 0.00000000 0.06329114   0.00000000 0.08860759
       Net     0.01265823 0.02531646  0.00000000 0.01265823 0.01265823 0.00000000 0.01265823 0.08860759   0.00000000 0.06329114
       outflow 0.01265823 0.07594937  0.02531646 0.00000000 0.02531646 0.00000000 0.01265823 0.08860759   0.01265823 0.08860759
              livestock$Region
livestock$Type    Somalia
       inflow  0.07594937
       Net     0.02531646
       outflow 0.03797468
```

```
> #percentage of prop table
> prop.table(TableA) *100
              livestock$Region
livestock$Type     Afar   Amhara Benishangul Dire Dawa  Gambela   Harari   Oromia  Oromiya Sidama_total     SNNP  Somalia
       inflow  3.797468 5.063291    2.531646  0.000000 1.265823 1.265823 0.000000 6.329114     0.000000 8.860759 7.594937
       Net     1.265823 2.531646    0.000000  1.265823 1.265823 0.000000 1.265823 8.860759     0.000000 6.329114 2.531646
       outflow 1.265823 7.594937    2.531646  0.000000 2.531646 0.000000 1.265823 8.860759     1.265823 8.860759 3.797468
```

Frequency tables: row and column proportions

- `prop.table(t, 1)` gives row proportions; `prop.table(t, 2)` gives column proportions.

```
> # Row percentage of the  prop table is equivalent to 3/79 for afar inflow where 79 is the total
> prop.table(TableA, margin = 1) *100
              livestock$Region
livestock$Type      Afar    Amhara Benishangul Dire Dawa   Gambela    Harari    Oromia   Oromiya Sidama_total      SNNP   Somalia
       inflow  10.344828 13.793103    6.896552  0.000000  3.448276  3.448276  0.000000 17.241379     0.000000 24.137931 20.689655
       Net      5.000000 10.000000    0.000000  5.000000  5.000000  0.000000  5.000000 35.000000     0.000000 25.000000 10.000000
       outflow  3.333333 20.000000    6.666667  0.000000  6.666667  0.000000  3.333333 23.333333     3.333333 23.333333 10.000000
> #column percentage of the prop table
> prop.table(TableA, margin = 2) *100
              livestock$Region
livestock$Type      Afar    Amhara Benishangul Dire Dawa   Gambela    Harari    Oromia   Oromiya Sidama_total      SNNP   Somalia
       inflow   60.00000  33.33333    50.00000   0.00000  25.00000 100.00000   0.00000  26.31579      0.00000  36.84211  54.54545
       Net      20.00000  16.66667     0.00000 100.00000  25.00000   0.00000  50.00000  36.84211      0.00000  26.31579  18.18182
       outflow  20.00000  50.00000    50.00000   0.00000  50.00000   0.00000  50.00000  36.84211    100.00000  36.84211  27.27273
>
```

DISCUSSION

For a Region × Type table, when would you want row percentages and when column percentages?

Indexing a vector

- An **index** tells R to use only specific elements of an object. Indexing is the primary way to create a subset in base R.

```r
livestock$CATTLE[1:10]   # cattle column, rows 1 to 10
livestock$CATTLE[3:7]
```

```
> livestock$CATTLE[1:10]
 [1]  930591   62214  733077  100838  258475  761124 1972622  924892 1952374 1690086
```

```
> livestock$CATTLE[3:7]  # index cattle column from 3 to 7
[1]  733077  100838  258475  761124 1972622
```

Indexing a data frame: `[rows, columns]`

- Give **both** a row index and a column index. The row index comes **before** the comma, the column index **after**.

```r
livestock[1, 2:6]
```

SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES
<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1377823	3251744	NA	NA	103573	325483	NA	NA

Indexing: all rows or all columns

- Leave the column specification blank to get all columns of a row — or the row specification blank to get all rows of a column.

```r
livestock[1, ]    # first row, all columns
livestock[ , 8]   # all rows, eighth column
```

Zone	CATTLE	SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES	Region	year	Type	Geography
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>
Zone_1 (Awsiresu)	930591	1377823	3251744	NA	NA	103573	325483	NA	NA	Afar	2017	Net	1

Missing values: how many per column?

- `colSums(is.na(df))` counts the NAs in every column.

```r
colSums(is.na(livestock))
```

```
> #summary of NA with each variable
> colSums(is.na(livestock))
     Zone    CATTLE  SHEEP_1  GOATS_0   HORSES     MULES  DONKEYS   CAMELS  POULTRY BEEHIVES   Region      year      Type 
        0         0         1         0        26        23         2        48         5        12         0         0         0 
Geography 
        0 
```

Missing values: percentage per column


```r
colMeans(is.na(livestock)) * 100
```

```
> colMeans(is.na(livestock))*100
     Zone    CATTLE   SHEEP_1   GOATS_0    HORSES     MULES   DONKEYS    CAMELS   POULTRY  BEEHIVES    Region      year      Type
 0.000000  0.000000  1.265823  0.000000 32.911392 29.113924  2.531646 60.759494  6.329114 15.189873  0.000000  0.000000  0.000000
Geography
 0.000000
```

Missing values in a specific column


```r
sum(is.na(livestock$colname))          # count
mean(is.na(livestock$colname)) * 100   # percentage
```

```
> #Sum of NA for specific columns
> sum(is.na(livestock$HORSES))
[1] 26
```

Rows where a column is missing

- Combine `is.na()` with indexing to see the rows with a missing horse count.

```r
livestock[is.na(livestock$horses), ]
```

Zone	CATTLE	SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES	Region	year	Type	Geography
<chr>	*<dbl>*	*<dbl>*	*<dbl>*	*<dbl>*	*<dbl>*	*<dbl>*	*<dbl>*	*<dbl>*	*<dbl>*	*<chr>*	*<dbl>*	*<chr>*	*<dbl>*
Zone_1 (Awsiresu)	930591	1377823	3251744	NA	NA	103573	325483	NA	NA	Afar	2017	Net	1
Zone_2 (Kelbetiresu)	62214	413083	1376619	NA	NA	59030	123434	12821	18689	Afar	2016	inflow	4
Zone_3 (Gebiresu)	733077	1112492	1328252	NA	91	50378	324045	37825	NA	Afar	2016	inflow	3
Zone_5 (Hariresu)	258475	384293	1082975	NA	NA	38842	164875	12580	NA	Afar	2016	outflow	2
Waghimra	367985	196608	550710	NA	2645	122909	NA	398167	135226	Amhara	2018	Net	4
Oromiya Special Zone	322932	39203	211362	NA	NA	60037	10232	347721	7043	Amhara	2015	outflow	3
West Gonder	359982	21001	196862	NA	451	31256	222	328354	22254	Amhara	2015	outflow	3
West Hararghe	1194184	97897	970168	NA	1071	263721	68498	956898	98580	Oromiya	2017	Net	4
Kelem Wollega	514841	186207	138080	NA	4572	54134	NA	475056	114362	Oromiya	2016	inflow	4
Erer	510064	922390	1258626	NA	1239	131184	902900	476	26275	Somalia	2015	Net	3

ℹ 16 more rows
ℹ Use `print(n = ...)` to see more rows

Filling missing values with the mean

- Index the missing cells with `is.na()` and assign the column mean (computed with `na.rm = TRUE`).

```r
# Fill missing values in cattle column with the mean
livestock$cattle[is.na(livestock$cattle)] <- mean(livestock$cattle, na.rm = TRUE)

# Fill missing values in horses column with the mean
livestock$horses[is.na(livestock$horses)] <- mean(livestock$horses, na.rm = TRUE)
```

Filling missing values with the median or mode

- The **median** is more robust to extreme values; for categorical columns use the **mode** (most frequent value).
- Base R has no `Mode()` function — define one first (or use a package).

```r
livestock$cattle[is.na(livestock$cattle)] <- median(livestock$cattle, na.rm = TRUE)

Mode <- function(x) { ux <- unique(na.omit(x)); ux[which.max(tabulate(match(x, ux)))] }
livestock$GeoType[is.na(livestock$GeoType)] <- Mode(livestock$GeoType)
```

Removing rows with missing values: `na.omit()`

- If missing values do not affect the analysis, `na.omit()` removes every row that contains an `NA`.
- **Back up your data first** — you may want to reverse the step.

```R
livestock_complete <- na.omit(livestock)
```

```
> # Dropping the missing values
> data4<-livestock # Take a back up of your data
> colSums(is.na(data4)) # see the missing value summary before dropping
     Zone    CATTLE   SHEEP_1   GOATS_0    HORSES     MULES   DONKEYS    CAMELS   POULTRY  BEEHIVES    Region      year      Type
        0         0         1         0        26        23         2        48         5        12         0         0         0
Geography
        0
> df_clean<-na.omit(data4) # drop the missing value
> colSums(is.na(df_clean)) #check that all are removed
     Zone    CATTLE   SHEEP_1   GOATS_0    HORSES     MULES   DONKEYS    CAMELS   POULTRY  BEEHIVES    Region      year      Type
        0         0         0         0         0         0         0         0         0         0         0         0         0
Geography
        0
>
```

Dropping rows: by missing column, by condition, by position

- Negative indexes drop rows by position; ! negates a logical condition.

```r
# drop rows where horses is NA
df_clean <- livestock[!is.na(livestock$horses), ]

# keep only rows with cattle <= 30000
df_new <- livestock[!(livestock$cattle > 30000), ]

df_new <- df[-c(2, 4), ]    # remove rows 2 and 4
df_new <- df[-3, ]          # remove row 3
df_new <- df[-(5:10), ]     # remove rows 5 to 10
```



DISCUSSION

15 min

Mean, median, mode — or drop?

1. A zone is missing its 2016 sheep count. Would you fill it with the regional mean, the zone's median across years, or leave it as NA? Why?

2. When is dropping rows with missing values acceptable, and when does it bias the results?

3. How would you document imputation decisions so users of your report understand them?

How to run it: No single right answer; steer toward 'depends on the missingness mechanism and the use of the indicator'.

TAKE A BREAK

Lunch Break



12:30 – 13:30 · 60 minutes

UP NEXT

Subsetting, new columns, sorting and the dplyr verbs

Logical expressions

- A logical expression asks R whether a statement is true and returns `TRUE` or `FALSE`.
- Test equality with `==`; other operators are `>`, `>=`, `<`, `<=`, `!` (not), `&` (and), `|` (or).

```R
livestock$CATTLE[1] == 11242
## [1] FALSE
livestock$CATTLE[1] == 930591
## [1] TRUE
```

Subsetting rows with a condition

- Put a logical expression in the **row** position of the index to keep only the rows where it is TRUE.
- Example: select only livestock **inflow** records.

```r
filtered <- livestock[livestock$Type == "inflow", ]
```

```
> #Filter only livestock inflow from the dataset
> livestock[livestock$Type=="inflow",]
# A tibble: 29 × 14
   Zone                CATTLE SHEEP_1 GOATS_0 HORSES MULES DONKEYS CAMELS POULTRY BEEHIVES Region   year Type   Geography
   <chr>                <dbl>   <dbl>   <dbl>  <dbl> <dbl>   <dbl>  <dbl>   <dbl>    <dbl> <chr>   <dbl> <chr>      <dbl>
 1 Zone_2 (Kelbetiresu)  62214  413083 1376619     NA    NA   59030 123434   12821    18689 Afar     2016 inflow         4
 2 Zone_3 (Gebiresu)    733077 1112492 1328252     NA    91   50378 324045   37825       NA Afar     2016 inflow         3
 3 Zone_4 (Fentiresu)   100838  338140  587919     98    NA   24433  66140      NA       NA Afar     2017 inflow         4
 4 South Gonder        1972622 1162805  662987  39516 30363  425555     NA 1778314   121087 Amhara   2017 inflow         3
 5 South Wello         1952374 1538406 1390763  40250 30134  664223   2642 1965306   156088 Amhara   2017 inflow         3
 6 Awi                 1130053  511127   86229  96832 17530   96617     NA  974128   109442 Amhara   2014 inflow         1
 7 Central Gonder      2622612  713912 1105494  26785  8274  447511     NA 2380169   159951 Amhara   2017 inflow         4
 8 East Shewa          1095554  331079  515309  18667  3719  288294  17249  641995     1622 Oromiya  2016 inflow         2
 9 Arsi                2893856 1340779  809104 277770 17321  541801     NA 1465515    78935 Oromiya  2018 inflow         2
10 Bale                 950015  227173  343790  56725  3423  138295   9337  419600   277744 Oromiya  2017 inflow         3
# ℹ 19 more rows
# ℹ Use `print(n = ...)` to see more rows
```

Subsetting rows: a specific year


```r
Data2015 <- livestock[livestock$year == 2015, ]
```

```
> Data2015<-livestock[livestock$year==2015, ] # filters mothers with first pregnancy only
> head(Data2015)
# A tibble: 6 × 14
  Zone                CATTLE SHEEP_1 GOATS_0 HORSES MULES DONKEYS CAMELS POULTRY BEEHIVES Region   year Type     Geography
  <chr>                <dbl>   <dbl>   <dbl>  <dbl> <dbl>   <dbl>  <dbl>   <dbl>    <dbl> <chr>   <dbl> <chr>        <dbl>
1 West Gojjam        3003155 1653139  303838  31268 70698  446980     NA 3282574   208195 Amhara   2015 outflow          4
2 Oromiya Special Zone  322932   39203  211362     NA    NA   60037  10232  347721     7043 Amhara   2015 outflow          3
3 West Gonder           359982   21001  196862     NA   451   31256    222  328354    22254 Amhara   2015 outflow          3
4 East Wollega         1534595  374749  229131   6701  6445  185449     NA 1013542   253475 Oromiya  2015 Net              3
5 Ilu Ababor            941071  288861  172948  36952  4744   36089     NA  734073   338529 Oromiya  2015 Net              4
6 West Shewa           2277521 1006804  306714 302558  8721  298867     NA 1052189   210785 Oromiya  2015 Net              4
>
```

Subsetting columns by name

- Example: zones with their cattle count and year.

```r
Newdata <- livestock[ , c('Zone', 'CATTLE', 'year')]
```

	Zone	CATTLE	year
	<chr>	<dbl>	<dbl>
1	Zone_1 (Awsiresu)	930591	2017
2	Zone_2 (Kelbetiresu)	62214	2016
3	Zone_3 (Gebiresu)	733077	2016
4	Zone_4 (Fentiresu)	100838	2017
5	Zone_5 (Hariresu)	258475	2016
6	North Gonder	761124	2014

Subsetting against a computed value

- Example: zones with a cattle population below the average.

```r
belowaverage <- livestock[livestock$CATTLE < mean(livestock$CATTLE, na.rm = TRUE), ]
```

```
> belowavarega <- livestock[livestock$CATTLE < mean(livestock$CATTLE,na.rm = TRUE),]
> head(belowavarega)
# A tibble: 6 × 14
  Zone                CATTLE SHEEP_1 GOATS_0 HORSES MULES DONKEYS CAMELS POULTRY BEEHIVES Region  year Type    Geography
  <chr>                <dbl>   <dbl>   <dbl>  <dbl> <dbl>   <dbl>  <dbl>   <dbl>    <dbl> <chr>  <dbl> <chr>       <dbl>
1 Zone_2 (Kelbetiresu)  62214  413083 1376619     NA    NA   59030 123434   12821    18689 Afar    2016 inflow          4
2 Zone_3 (Gebiresu)    733077 1112492 1328252     NA    91   50378 324045   37825       NA Afar    2016 inflow          3
3 Zone_4 (Fentiresu)   100838  338140  587919     98    NA   24433  66140      NA       NA Afar    2017 inflow          4
4 Zone_5 (Hariresu)    258475  384293 1082975     NA    NA   38842 164875   12580       NA Afar    2016 outflow         2
5 North Gonder         761124  507316  557070  68943 13642  209833     NA 1062025   168138 Amhara  2014 Net             3
6 Waghimra             367985  196608  550710     NA  2645  122909     NA  398167   135226 Amhara  2018 Net             4
```

Logical connectors: OR (|)

- Example: livestock data for 2014 **or** 2015.

```r
Data1415 <- livestock[livestock$year == 2014 | livestock$year == 2015, ]
```

```
> Data1415<-livestock[livestock$year==2014 | livestock$year==2015,]
> head(Data1415)
# A tibble: 6 × 14
  Zone                CATTLE SHEEP_1 GOATS_0 HORSES MULES DONKEYS CAMELS POULTRY BEEHIVES Region  year Type     Geography
  <chr>                <dbl>   <dbl>   <dbl>  <dbl> <dbl>   <dbl>  <dbl>   <dbl>    <dbl> <chr>  <dbl> <chr>        <dbl>
1 North Gonder        761124  507316  557070  68943 13642  209833     NA 1062025   168138 Amhara  2014 Net              3
2 North Wello         924892  928259  583226  37547 16326  293698  14847 1069976    48456 Amhara  2014 outflow          3
3 West Gojjam        3003155 1653139  303838  31268 70698  446980     NA 3282574   208195 Amhara  2015 outflow          4
4 Awi                1130053  511127   86229  96832 17530   96617     NA  974128   109442 Amhara  2014 inflow           1
5 Oromiya Special Zone  322932   39203  211362     NA    NA   60037  10232  347721     7043 Amhara  2015 outflow          3
6 West Gonder           359982   21001  196862     NA   451   31256    222  328354    22254 Amhara  2015 outflow          3
```

Logical connectors: AND (&)

- Example: zone data for Oromia **and** the year 2018.

```r
OromiaZone2018 <- livestock[livestock$Region == 'Oromiya' & livestock$year == 2018, ]
```

```
# A tibble: 2 × 14
  Zone       CATTLE SHEEP_1 GOATS_0 HORSES MULES DONKEYS CAMELS POULTRY BEEHIVES Region   year Type    Geography
  <chr>       <dbl>   <dbl>   <dbl>  <dbl> <dbl>   <dbl>  <dbl>   <dbl>    <dbl> <chr>   <dbl> <chr>       <dbl>
1 Arsi      2893856 1340779  809104 277770 17321  541801     NA 1465515    78935 Oromiya  2018 inflow          2
2 East Bale  499570   65951  455202    237   979  122598  81256  115276   170348 Oromiya  2018 Net             1
```

Replacing values in a column

- Two spellings of the same region appear because of a typo. Index the wrong values and assign the correct label.

```r
livestock$Region[livestock$Region == 'Oromia'] <- 'Oromiya'
```

```
> table(livestock$Region)

        Afar       Amhara  Benishangul     Dire Dawa      Gambela       Harari       Oromia      Oromiya Sidama_total         SNNP
           5           12            4            1            4            1            2           19            1           19
     Somalia
          11
```

KEY POINT

Standardise labels before frequency tables, joins and dashboards — otherwise one region silently becomes two.



HANDS-ON EXERCISE

15 min

Exercise: subsetting the livestock data

1. Filter camel data for Somali and Afar only.
2. Filter sheep and goat data for the year 2018.
3. Filter outflow livestock data for each year and compare the trends.

Creating new columns

- Assign to a new column name with $ to create it; the calculation applies to every row.

```r
# sheep-to-goat ratio
livestock$Sheep2Goat <- livestock$sheep / livestock$goat

# a running index column
livestock$index <- 1:nrow(livestock)
```

Creating a category from an existing column

- Example: a `CattleRate` category based on the cattle population, using conditional indexing.

```
# Creating new column for categorizing zone Cattle population  as lowest, medium , high, highest
livestock1$CattleRate<- NA
livestock1$CattleRate[livestock1$CATTLE <=8036] <- "Lowest"
livestock1$CattleRate[livestock1$CATTLE > 8036 & livestock1$CATTLE < 838746.7] <- "Medium"
livestock1$CattleRate[livestock1$CATTLE > 838746.7 & livestock1$CATTLE <2803155] <- "High"
livestock1$CattleRate[livestock1$CATTLE > 2803155 ] <- "Highest"
colnames(livestock)
```

	Zone	CATTLE	SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES	Region	year	Type	Geography	CattleRate	GeoType
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>	<chr>	<fct>
1	Zone_1 (Awsiresu)	930591	1377823	3251744	NA	NA	103573	325483	NA	NA	Afar	2017	Net	1	High	Low la…
2	Zone_2 (Kelbetiresu)	62214	413083	1376619	NA	NA	59030	123434	12821	18689	Afar	2016	infl…	4	Medium	Mounta…
3	Zone_3 (Gebiresu)	733077	1112492	1328252	NA	91	50378	324045	37825	NA	Afar	2016	infl…	3	Medium	High l…
4	Zone_4 (Fentiresu)	100838	338140	587919	98	NA	24433	66140	NA	NA	Afar	2017	infl…	4	Medium	Mounta…
5	Zone_5 (Hariresu)	258475	384293	1082975	NA	NA	38842	164875	12580	NA	Afar	2016	outf…	2	Medium	Midland
6	North Gonder	761124	507316	557070	68943	13642	209833	NA	1062025	168138	Amhara	2014	Net	3	Medium	High l…

Creating categories with `cut()`

- `cut()` bins a numeric column into intervals and labels each interval in one call.

```
livestock1$CattleRate1<-cut(livestock1$CATTLE, breaks = c(0,8036,838746.7,2803155,3003156),
                labels = c("Lowest", "Midium", "High yield","Highest yield"),
                include.lowest = T, right = F )
```

Factor categorical variables

- Convert numeric codes to text categories: the Geography column coded 1–4 becomes Lowland, Midland, Highland, Mountainous.

```
livestock1$GeoType<-NA
livestock1$GeoType<- factor(livestock1$Geography,levels = 1:4,labels = c("Low land", "Midland", "High land","Mountanous"))
head(livestock1)
```

```
# A tibble: 6 × 16
```

	Zone	CATTLE	SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES	Region	year	Type	Geography	CattleRate	GeoType
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>	<chr>	<fct>
1	Zone_1 (Awsiresu)	930591	1377823	3251744	NA	NA	103573	325483	NA	NA	Afar	2017	Net	1	High	Low la…
2	Zone_2 (Kelbetiresu)	62214	413083	1376619	NA	NA	59030	123434	12821	18689	Afar	2016	infl…	4	Medium	Mounta…
3	Zone_3 (Gebiresu)	733077	1112492	1328252	NA	91	50378	324045	37825	NA	Afar	2016	infl…	3	Medium	High l…
4	Zone_4 (Fentiresu)	100838	338140	587919	98	NA	24433	66140	NA	NA	Afar	2017	infl…	4	Medium	Mounta…
5	Zone_5 (Hariresu)	258475	384293	1082975	NA	NA	38842	164875	12580	NA	Afar	2016	outf…	2	Medium	Midland
6	North Gonder	761124	507316	557070	68943	13642	209833	NA	1062025	168138	Amhara	2014	Net	3	Medium	High l…

Dropping columns

- Set a column (or a set of columns) to `NULL` to remove it.

```R
livestock$GeoType <- NULL                          # drops one column
livestock[ , c('Geography', 'GeoType')] <- NULL  # drops several
```

Sorting data with `order()`

- Sort by one or more columns — here Region, then Cattle, ascending by default. Use `-` or `decreasing = TRUE` for descending.

```r
sorted2 <- livestock[order(livestock$Region, livestock$CATTLE), ]
```

	Zone	CATTLE	SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES	Region	year	Type	Geography	CattleRate	GeoType
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>	<chr>	<fct>
1	Zone_2 (Kelbetiresu)	62214	413083	1376619	NA	NA	59030	123434	12821	18689	Afar	2016	infl…	4	Medium	Mounta…
2	Zone_4 (Fentiresu)	100838	338140	587919	98	NA	24433	66140	NA	NA	Afar	2017	infl…	4	Medium	Mounta…
3	Zone_5 (Hariresu)	258475	384293	1082975	NA	NA	38842	164875	12580	NA	Afar	2016	outf…	2	Medium	Midland
4	Zone_3 (Gebiresu)	733077	1112492	1328252	NA	91	50378	324045	37825	NA	Afar	2016	infl…	3	Medium	High l…
5	Zone_1 (Awsiresu)	930591	1377823	3251744	NA	NA	103573	325483	NA	NA	Afar	2017	Net	1	High	Low la…
6	Oromiya Special Zone	322932	39203	211362	NA	NA	60037	10232	347721	7043	Amhara	2015	outf…	3	Medium	High l…

HANDS-ON EXERCISE

Create a new column with the sum of all livestock, and a second column rating poultry production in three categories.

DAY 2 · MODULE 5

05

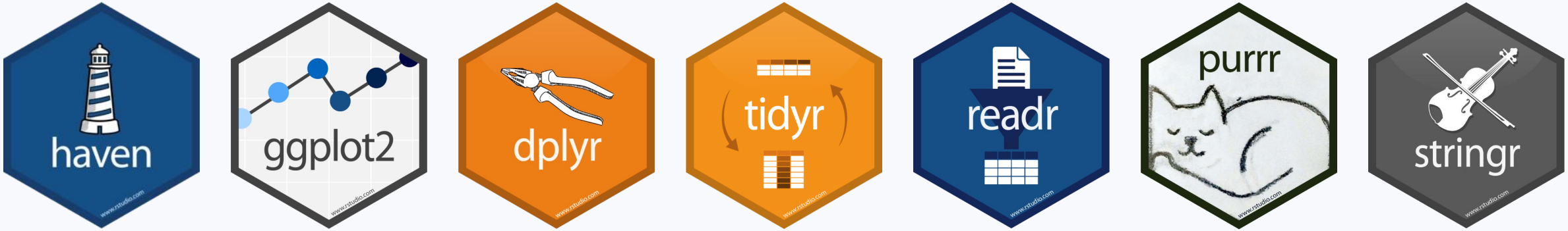
The tidyverse: dplyr & Combining Data

A consistent grammar for manipulating data — and joining datasets from several sources.

In this module you will

- Describe the tidyverse philosophy and its core packages
- Use the six dplyr verbs
- Chain operations with the pipe `%>%`
- Summarise by group
- Append with `rbind()` / `cbind()` and join with `merge()`

The tidyverse: a family of packages



Eight core packages, one shared grammar — loaded together with library(tidyverse).

Why the tidyverse?

- A hugely popular collection of packages developed by **Hadley Wickham** and the Posit team — the modern engine for data science in R.
- It is an **'opinionated'** collection: every package shares the same design philosophy, grammar and data structures, built around the idea of **tidy data**.
- Because they speak the same language, you pass data seamlessly from one package to the next with the **pipe** (`%>%` or `|>`).
- They add a lot of functionality missing in base R while remaining consistent.

The eight core packages

- `library(tidyverse)` loads eight packages at once, each covering a phase of the data-science workflow.

Package	The Core Purpose	What it handles
`readr`	**Import**	Reading flat data files (like `.csv` or `.tsv`) into R quickly and cleanly.
`tibble`	**Structure**	A modern, lazy, and strict reimagining of the traditional R `data.frame`.
`tidyr`	**Tidy**	Reshaping your data layouts (e.g., pivoting data from wide to long formats, handling missing values).
`dplyr`	**Manipulate**	The data manipulation workhorse (where your `select()`, `filter()`, `mutate()`, and `summarize()` functions live).
`ggplot2`	**Visualize**	Building beautiful, layered data visualizations based on the "Grammar of Graphics."

dplyr: six verbs

filter()

Keep rows that match conditions.

select()

Pick columns by name.

arrange()

Sort rows.

mutate()

Add or change columns.

summarise()

Collapse values into summary statistics.

group_by()

Perform operations by group.

`filter()`: rows matching a condition

- Example: livestock information for Oromiya only.

```r
library(tidyverse)
Oromiya <- filter(livestock, Region == "Oromiya")
```

Zone	CATTLE	SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES	Region	year	Type	Geography
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>
West Wollega	1772949	588106	91780	6686	3132	251512	NA	1605371	561317	Oromiya	2014	outflow	2
East Wollega	1534595	374749	229131	6701	6445	185449	NA	1013542	253475	Oromiya	2015	Net	3
Ilu Ababor	941071	288861	172948	36952	4744	36089	NA	734073	338529	Oromiya	2015	Net	4
Jimma	2629417	818781	483967	69948	9432	185772	NA	1633085	582202	Oromiya	2014	Net	1
West Shewa	2277521	1006804	306714	302558	8721	298867	NA	1052189	210785	Oromiya	2015	Net	4
North Shewa	1317105	905714	102364	90140	NA	318792	NA	570560	15895	Oromiya	2015	outflow	2

`filter()` with logical connectors

- Example: Oromiya data for years after 2016.

```r
Oro2016 <- filter(livestock, Region == "Oromiya" & year > 2016)
head(Oro2016)
```

```
# A tibble: 6 × 16
  Zone           CATTLE SHEEP_1 GOATS_0 HORSES MULES DONKEYS CAMELS POULTRY BEEHIVES Region   year Type    Geography CattleRate GeoType
  <chr>           <dbl>   <dbl>   <dbl>  <dbl> <dbl>   <dbl>  <dbl>   <dbl>    <dbl> <chr>   <dbl> <chr>       <dbl> <chr>      <fct>
1 Arsi          2893856 1340779  809104 277770 17321  541801     NA 1465515    78935 Oromiya  2018 inflow          2 Highest    Midland
2 West Hararghe 1194184   97897  970168     NA  1071  263721  68498  956898    98580 Oromiya  2017 Net             4 High       Mountanous
3 Bale           950015  227173  343790  56725  3423  138295   9337  419600   277744 Oromiya  2017 inflow          3 High       High land
4 Borena         578490  231097  406328    201   291   48842  43148  111561    10165 Oromiya  2017 Net             4 Medium     Mountanous
5 West Arsi     1812955  663458  200398 155592  3152  258165     NA  624386    57177 Oromiya  2017 outflow         3 High       High land
6 Buno Bedele    469186  121488   90515  11877  1418   30568     NA  235294    63919 Oromiya  2017 outflow         4 Medium     Mountanous
>
```

Chaining with the pipe %>%

- dplyr lets us string functions together: the object on the left becomes the first argument of the function on the right.
- Simple here — but this is what makes complex manipulations quick and readable.

```r
livestock %>% filter(Region == "Oromiya" & year > 2015)
```

Zone	CATTLE	SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES	Region	year	Type	Geography
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>
East Shewa	1095554	331079	515309	18667	3719	288294	17249	641995	1622	Oromiya	2016	inflow	2
Arsi	2893856	1340779	809104	277770	17321	541801	NA	1465515	78935	Oromiya	2018	inflow	2
West Hararghe	1194184	97897	970168	NA	1071	263721	68498	956898	98580	Oromiya	2017	Net	4
East Hararge	1220872	275365	1189842	1388	NA	296621	11431	1583267	59817	Oromiya	2016	outflow	1
Bale	950015	227173	343790	56725	3423	138295	9337	419600	277744	Oromiya	2017	inflow	3
Borena	578490	231097	406328	201	291	48842	43148	111561	10165	Oromiya	2017	Net	4
West Arsi	1812955	663458	200398	155592	3152	258165	NA	624386	57177	Oromiya	2017	outflow	3
Kelem Wollega	514841	186207	138080	NA	4572	54134	NA	475056	114362	Oromiya	2016	inflow	4
East Bale	499570	65951	455202	237	979	122598	81256	115276	170348	Oromiya	2018	Net	1

HANDS-ON EXERCISE

Use filter() to select livestock inflow for Somali for the year 2018.

`select()`: columns by name or range

- Select variables by name, by range with `:`, or with helper functions such as `starts_with()`.

```r
select(livestock, Region:CAMELS)   # columns from Region to CAMELS
select(livestock, Region:CATTLE)   # columns from Region to CATTLE

TypeYear <- livestock %>% select(CATTLE, year, Type)
View(TypeYear)
```

	CATTLE	year	Type
1	930591	2017	Net
2	62214	2016	inflow
3	733077	2016	inflow
4	100838	2017	inflow
5	258475	2016	outflow
6	761124	2014	Net
7	1972622	2017	inflow
8	924892	2014	outflow
9	1952374	2017	inflow
10	1952374	2017	outflow
11	2185995	2016	outflow
12	3003155	2015	outflow

Combining `filter()` and `select()`

- Cattle, sheep and inflow for Oromia in a specific year.

	CATTLE	SHEEP_1	year	Type	Region
	<dbl>	<dbl>	<dbl>	<chr>	<chr>
1	1095554	331079	2016	inflow	Oromiya
2	2893856	1340779	2018	inflow	Oromiya
3	950015	227173	2017	inflow	Oromiya
4	1542178	552133	2014	inflow	Oromiya
5	514841	186207	2016	inflow	Oromiya

```
oromiyinflow<-livestock %>% filter(Region=="Oromiya" & Type=='inflow') %>% select(CATTLE,SHEEP_1,year,Type,Region)
View(oromiyinflow)
```

arrange(): sorting rows

- Order the data frame by one or more columns — here by Region.

```
data2<-livestock %>% select(Region,year,Type,Geography) %>% arrange(Region)
head(data2)
```

	Region	year	Type	Geography
	<chr>	<dbl>	<chr>	<dbl>
1	Afar	2017	Net	1
2	Afar	2016	inflow	4
3	Afar	2016	inflow	3
4	Afar	2017	inflow	4
5	Afar	2016	outflow	2
6	Amhara	2014	Net	3
7	Amhara	2017	inflow	3
8	Amhara	2014	outflow	3
9	Amhara	2017	inflow	3
0	Amhara	2017	outflow	3

arrange(desc()): descending order

```
data3<-livestock %>% select(Region,year,Type,Geography) %>% arrange(desc(Region))
head(data3)
```

	Region	year	Type	Geography
	<chr>	<dbl>	<chr>	<dbl>
1	Somalia	2015	inflow	2
2	Somalia	2014	inflow	2
3	Somalia	2014	outflow	3
4	Somalia	2015	Net	3
5	Somalia	2014	inflow	1
6	Somalia	2015	inflow	1
7	Somalia	2014	outflow	2
8	Somalia	2014	inflow	4
9	Somalia	2016	Net	4
10	Somalia	2016	inflow	1

Exercise: least and most cattle

- Which Region, in which year, had the **least** cattle? And the **most**? (Hint: re-sort in descending order, or use `tail()`.)

```R
livestock %>% select(Region, year, cattle) %>% arrange(cattle) %>% head(n = 1)
livestock %>% select(Region, year, cattle) %>% arrange(cattle) %>% tail(n = 1)
```

UNIVERSITY of WASHINGTON

`mutate()`: add or replace columns

- Example: cattle-to-sheep ratio as a new column.

```r
livestock <- livestock %>% mutate(cattle_sheep = cattle / sheep)
```

	Zone	CATTLE	SHEEP_1	GOATS_0	HORSES	MULES	DONKEYS	CAMELS	POULTRY	BEEHIVES	Region	year	Type	Geography	CATTLE_SHEEP
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>	<dbl>
1	Zone_1 (Awsiresu)	930591	1377823	3251744	NA	NA	103573	325483	NA	NA	Afar	2017	Net	1	0.675
2	Zone_2 (Kelbetiresu)	62214	413083	1376619	NA	NA	59030	123434	12821	18689	Afar	2016	inflow	4	0.151
3	Zone_3 (Gebiresu)	733077	1112492	1328252	NA	91	50378	324045	37825	NA	Afar	2016	inflow	3	0.659
4	Zone_4 (Fentiresu)	100838	338140	587919	98	NA	24433	66140	NA	NA	Afar	2017	inflow	4	0.298
5	Zone_5 (Hariresu)	258475	384293	1082975	NA	NA	38842	164875	12580	NA	Afar	2016	outflow	2	0.673
6	North Gonder	761124	507316	557070	68943	13642	209833	NA	1062025	168138	Amhara	2014	Net	3	1.50
7	South Gonder	1972622	1162805	662987	39516	30363	425555	NA	1778314	121087	Amhara	2017	inflow	3	1.70
8	North Wello	924892	928259	583226	37547	16326	293698	14847	1069976	48456	Amhara	2014	outflow	3	0.996
9	South Wello	1952374	1538406	1390763	40250	30134	664223	2642	1965306	156088	Amhara	2017	inflow	3	1.27
10	North Shewa	1690086	1770234	844706	97838	10081	606883	11242	1660325	67901	Amhara	2017	outflow	3	0.955

ℹ 69 more rows

summarise(): a data frame of summary statistics

```R
livestock %>% summarise(mean_cattle = mean(cattle))

livestock %>% summarise(meanCattle  = mean(cattle),
                        totalCattle = sum(cattle))
```

HANDS-ON EXERCISE

Chain filter() and summarise() to find the min, mean and max sheep population for 2018.

group_by(): operations by group

- Converts a data frame into a **grouped** one, so the following `summarise()` works per group.
- Example: min, max, mean and sum of cattle by year.

```
sumarr2<-livestock %>%
  group_by(year) %>%
  summarize(minimum=min(CATTLE), maximum=max(CATTLE), meanCattle=mean(CATTLE), total=sum(CATTLE))
```

year	minimum	maximum	meanCattle	total
<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
2014	77935	2629417	1055916.	17950565
2015	7636	3003155	818419.	16368386
2016	30542	2185995	544069.	9793239
2017	44448	2622612	1037268.	14521746
2018	20171	2893856	762705.	7627051



HANDS-ON EXERCISE

15 min

Exercise: grouped summaries

1. Chain `filter()`, `group_by()` and `summarise()` to find the min, mean and max sheep population **by region** for 2016.

2. Repeat for goats, but group by Region **and** Type.

3. Which region has the largest gap between its minimum and maximum zone-level sheep count?

How to run it: livestock %>% filter(year == 2016) %>% group_by(Region) %>% summarise(min = min(sheep), mean = mean(sheep), max = max(sheep))

TAKE A BREAK

Tea Break

15:00 – 15:15 · 15 minutes

UP NEXT

Appending and merging datasets



Appending rows and columns: `rbind()` and `cbind()`

- `rbind()` combines objects **as rows**; `cbind()` combines them **as columns**; `c()` flattens into a single vector.

R

```
rbind(1:4, 5:8)        # combine as rows
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    5    6    7    8

cbind(1:4, 5:8)        # combine as columns -> 4 rows x 2 columns

c(1:4, 5:8)            # combine into a single vector
## [1] 1 2 3 4 5 6 7 8
```

`rbind()` and `cbind()` with data frames: caveats

- They can merge or append data frames, **but**:
  - `rbind()` requires all columns in both data frames to have the **same names**, and all to be present.
  - `cbind()` requires the **same number of rows**, sorted in the **same order**.
- Put simply, these are not smart functions — they do no housekeeping for you.

Appending rows: 2014 with 2015


```
 #Filter rows  with year 2014 and 2015
data2014<-filter(livestock, year==2014)
data2015<-filter(livestock, year==2015)
nrow(data2014)
nrow(data2015)

# bind data2014 and  data2015 using rbinf
appended<-rbind(data2014,data2015)
view(appended)
nrow(appended)
```

Appending columns with `cbind()`

- Use only when you are certain the columns have the same number of observations and sort order — mostly for building test data or combining function outputs.

```
# column binding: bind column cattle with column year
yearcol<-livestock[,'year']
Cattlcol<-livestock[,'CATTLE']
coldata<-cbind(yearcol,Cattlcol)
```

Appending rows: labelling the columns


```
# Read the two files and append by row
FileOne<- read_excel("All_LivestockFileOne.xlsx")
FileTwo<-read_excel("All_LivestockFileTwo.xlsx")

print(names(FileOne)) # check columns before combining
print(nrow(FileOne)) # check number  of  rows before combining
print(names(FileTwo))
print(nrow(FileTwo))
head(FileOne)
head(FileTwo)
```

Appending rows: checking the result


```
> FileOne
# A tibble: 43 × 11
   Zones                CATTLE    SHEEP    GOATS HORSES  MULES   DONKEYS CAMELS  POULTRY  BEEHIVES Region
   <chr>                 <dbl>    <dbl>    <dbl> <chr>   <chr>     <dbl> <chr>   <chr>    <chr>    <chr>
 1 NA                       NA       NA       NA NA      NA           NA NA      NA       NA       NA
 2 National           66260987 38013272 45716091 2138928 357743 10020518 6978053 41350684 5982336  National
 3 Afar                2085194  3625830  7627510 98      91       276257 1003978 63227    18689    Afar_Total
 4 Zone_1 (Awsiresu)    930591  1377823  3251744 -       -        103573 325483  -        -        Afar
 5 Zone_2 (Kelbetiresu)  62214   413083  1376619 -       -         59030 123434  12821    18689    Afar
 6 Zone_3 (Gebiresu)    733077  1112492  1328252 -       91        50378 324045  37825    -        Afar
 7 Zone_4 (Fentiresu)   100838   338140   587919 98      -         24433 66140   -        -        Afar
 8 Zone_5 (Hariresu)    258475   384293  1082975 -       -         38842 164875  12580    -        Afar
 9 Amhara             17293813 10267358 6808301 518513  218858  3879122 39184   16334631 1330300  Amhara_Total
10 North Gonder         761124   507316   557070 68943   13642    209833 -       1062025  168138   Amhara
# ℹ 33 more rows
```

```
> FileTwo
# A tibble: 46 × 11
   Zones    CATTLE SHEEP       GOATS HORSES MULES DONKEYS CAMELS  POULTRY BEEHIVES Region
   <chr>     <dbl> <chr>       <dbl> <chr>  <chr> <chr>   <chr>   <chr>   <chr>    <chr>
 1 NA           NA NA             NA NA     NA    NA      NA      NA      NA       NA
 2 Somalie 5956828 10557274 16505748 -      1636  980960  5682572 383756  71681    Somalia_total
 3 Siti      50559 112658     468537 -      -     23754   12748   12700   -        Somalia
 4 Fafan    663783 746143     846829 -      214   81647   129449  36374   571      Somalia
 5 Jerer    779214 3205143   4778656 -      -     81811   1123079 -       -        Somalia
 6 Erer     510064 922390    1258626 -      1239  131184  902900  476     26275    Somalia
 7 Korahe   524803 860701     734998 -      -     61894   437208  -       -        Somalia
 8 Shebele  667603 577212    1052044 -      -     89103   250720  96387   -        Somalia
 9 Dollo    688007 791174     647253 -      -     29439   311381  183     -        Somalia
10 Afder    672711 1206300   3490406 -      -     176530  585291  19903   1143     Somalia
# ℹ 36 more rows
# ℹ Use `print(n = ...)` to see more rows
>
```

Appending rows: combined dataset

```
#Combine the two based on row
FileCombined<-rbind(FileOne,FileTwo) # Note that this does not work of number of columns are different
print(nrow(FileCombined))
```

```
> FileCombined
# A tibble: 89 × 11
   Zones                CATTLE SHEEP      GOATS HORSES MULES DONKEYS CAMELS POULTRY BEEHIVES Region
   <chr>                 <dbl> <chr>      <dbl> <chr>  <chr> <chr>   <chr>  <chr>   <chr>    <chr>
 1 NA                       NA NA            NA NA     NA    NA      NA     NA      NA       NA
 2 National           66260987 38013272 45716091 2138928 357743 10020518 6978053 41350684 5982336 National
 3 Afar                2085194 3625830   7627510 98      91    276257   1003978 63227    18689    Afar_Tota
 4 Zone_1 (Awsiresu)    930591 1377823   3251744 -       -     103573   325483  -        -        Afar
 5 Zone_2 (Kelbetiresu)  62214 413083    1376619 -       -     59030    123434  12821    18689    Afar
 6 Zone_3 (Gebiresu)    733077 1112492   1328252 -       91    50378    324045  37825    -        Afar
 7 Zone_4 (Fentiresu)   100838 338140    587919 98      -     24433    66140   -        -        Afar
 8 Zone_5 (Hariresu)    258475 384293    1082975 -       -     38842    164875  12580    -        Afar
 9 Amhara             17293813 10267358 6808301 518513  218858 3879122  39184   16334631 1330300  Amhara_To
10 North Gonder         761124 507316    557070 68943   13642 209833   -       1062025  168138   Amhara
# ℹ 79 more rows
# ℹ Use `print(n = ...)` to see more rows
>
```

Merging with `merge()`

- `merge()` is similar to *merge* in Stata: you need one or more **key variables present in both** data frames.
- Typical case: instead of one dataset you receive two files, each with its own variables plus the shared identifiers.

Options

- `by = c("id1", "id2")` — several identifiers; `by.x` / `by.y` — keys with different names.
- `all = FALSE` (default) keep only matches · `all = TRUE` keep everything · `all.x = TRUE` / `all.y = TRUE` keep all of one side.

```R
vipclsMerge2 <- merge(demogData, pregData, by.x = "patid", by.y = "id")
```

Merging: read the dairy data

- `livestock.xlsx` and `dairy_data.xlsx` share **zone** as the common attribute.

```
> #======Dairy Data=====
> dairyData<- read_excel("dairy_data.xlsx")

> colnames(dairyData)
 [1] "Zone"                          "Number_of_Dairy_cows"          "Number_of_milking_cows"        "Average_daily_Milk_production"
 [5] "Average_Lactation_period"      "Total_Milk _production"        "Number_of_Dairy_camels"        "Number_of_milking_camels"
 [9] "Average_daily_milk_production" "Average _lactation_period"     "Total_Milk_production"
```

Merging: standardise the key first

- Before merging, make the zone values identical in both datasets.

```
# Zones only in livestock but  not in dairy data
setdiff(unique(livestock$Zone),unique(dairyData$Zone))
# Zones only in dairy data but  not in livestock
setdiff(unique(dairyData$Zone),unique(livestock$Zone))
```

Merging: merge on zone and check the columns

```
# After standardizing, merge the two data based on the zone
Livestock_dairy<-merge(livestock,dairyData, by=c("Zone"), all=TRUE)
```

```
> colnames(Livestock_dairy)
 [1] "Zone"                          "CATTLE"                        "SHEEP_1"                       "GOATS_0"
 [5] "HORSES"                        "MULES"                         "DONKEYS"                       "CAMELS"
 [9] "POULTRY"                       "BEEHIVES"                      "Region"                        "year"
[13] "Type"                          "Geography"                     "CATTLE_SHEEP"                  "Number_of_Dairy_cows"
[17] "Number_of_milking_cows"        "Average_daily_Milk_production" "Average_Lactation_period"      "Total_Milk _production"
[21] "Number_of_Dairy_camels"        "Number_of_milking_camels"      "Average_daily_milk_production" "Average _lactation_period"
[25] "Total_Milk_production"
```



DISCUSSION

How would you check that no zones were silently dropped by the merge? Which merge option would keep them?



HANDS-ON EXERCISE

30 min

Exercise: build the merged livestock–dairy dataset

1. Read the livestock and dairy files. Compare the unique zone names and standardise any mismatches.

2. Merge on zone. How many rows do you get with the default, with `all.x = TRUE`, and with `all = TRUE`? Explain the differences.

3. Save the merged result with `write.csv(merged, 'MergedLivestock.csv', row.names = FALSE)` and reload it to confirm.

How to run it: The row-count comparison across merge options is the key learning moment — ask each pair to report their numbers.



DISCUSSION

15 min

Day 2 reflection

1. Base R indexing or dplyr verbs — which felt more natural, and why?
2. Which data-preparation step did you find hardest today?
3. Bring one question tomorrow that you would like a chart or dashboard to answer.

How to run it: Remind participants to save their scripts; tomorrow builds on the merged livestock data frame.

TRAINING DAY

3

Sunday, 20 September 2026

Visualisation with ggplot2, reproducible reports and dashboards

Today's schedule

08:30 – 09:00	Recap of Day 2 & questions
09:00 – 10:30	Module 6 · ggplot2: grammar of graphics, scatter and line plots
10:30 – 10:45	**Tea break**
10:45 – 12:30	Bar charts · Module 7 · R Markdown and reproducible reports
12:30 – 13:30	**Lunch break**
13:30 – 15:00	Quarto and Quarto dashboards
15:00 – 15:15	**Tea break**
15:15 – 16:15	Hands-on: build your own dashboard
16:15 – 17:00	Post-assessment, key takeaways & closing



DISCUSSION

15 min

Recap of Day 2

1. What does `na.rm = TRUE` do, and why is it needed so often?

2. Explain to your neighbour, word by word, what `livestock %>% filter(year == 2016) %>% group_by(Region) %>% summarise(mean(CATTLE))` does.

3. What was the row-count difference between your default merge and `all.x = TRUE` — and why?

DAY 3 · MODULE 6

06

Data Visualisation with ggplot2

Building charts layer by layer with the grammar of graphics.

In this module you will

- Explain geoms, aesthetics, scales, facets, themes and labels
- Draw scatter plots and add colour, size, titles and smoothing
- Draw line charts of trends over years
- Draw simple, clustered and stacked bar charts
- Combine dplyr pipelines with ggplot2

ggplot2: a small set of building blocks

ggplot2 builds complex graphics from a few components, added together with +. The online reference (ggplot2.tidyverse.org/reference) is organised by these blocks.

Geoms

The plot type: geom_point(), geom_line(), geom_col(), geom_boxplot(), geom_histogram(), geom_density(), geom_text() …

Aesthetics · aes()

Map variables to visual properties: x, y, colour, fill, size, shape, linetype, alpha, label.

Scales

Control how data values map to visuals — axis limits, colour palettes, log scales.

Facets

Split the data into multiple panels, e.g. one per Region.

Themes

Control the overall appearance and style (theme_minimal(), fonts, grid lines).

Labels · labs()

Titles, subtitles, captions and axis labels.

Aesthetics: mapping data to features of a geom

- Aesthetics map elements of the **data** to features of a **geom**. Specify them with `aes()`.
- Not all aesthetics are meaningful for all geoms — *shape* makes sense for points but not for lines.

Common aesthetics

– `x`, `y` position · `label` · `colour` (border) · `fill` (interior) · `shape` · `linetype` · `size` · `alpha` (transparency)



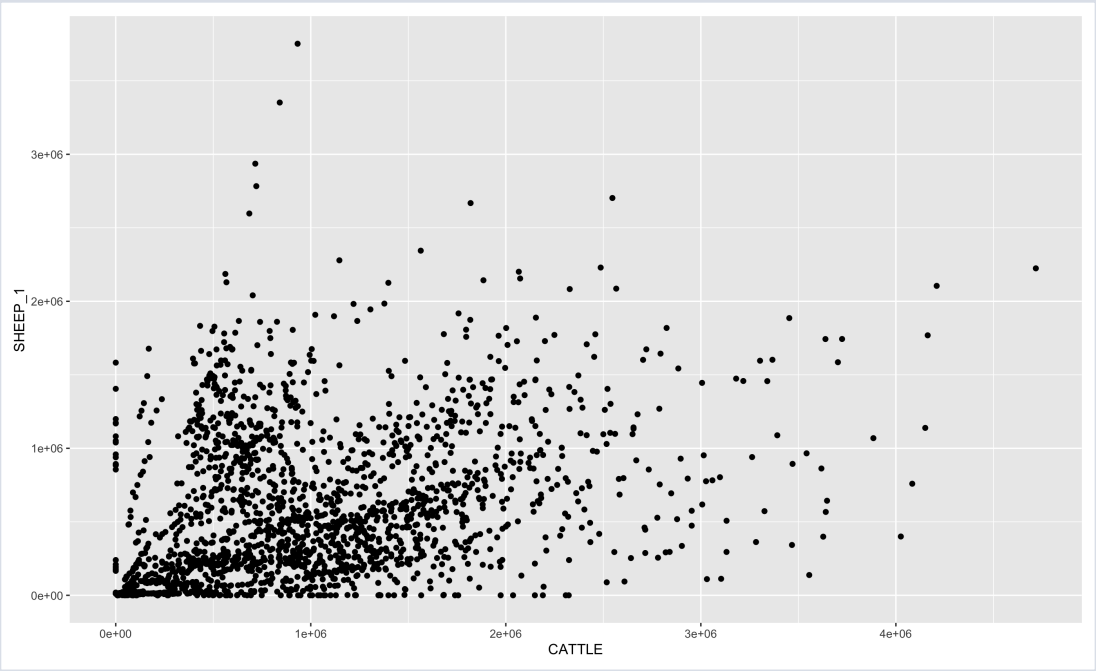
DISCUSSION

You want to show cattle vs sheep for five years and nine regions on one chart. Which aesthetics would you use for year and region?

Scatter plot: basic syntax

- `ggplot()` initialises the plot, names the data and sets the default aesthetics; `+ geom_point()` adds the points.
- Here CATTLE is mapped to the x-axis and SHEEP_1 to the y-axis.

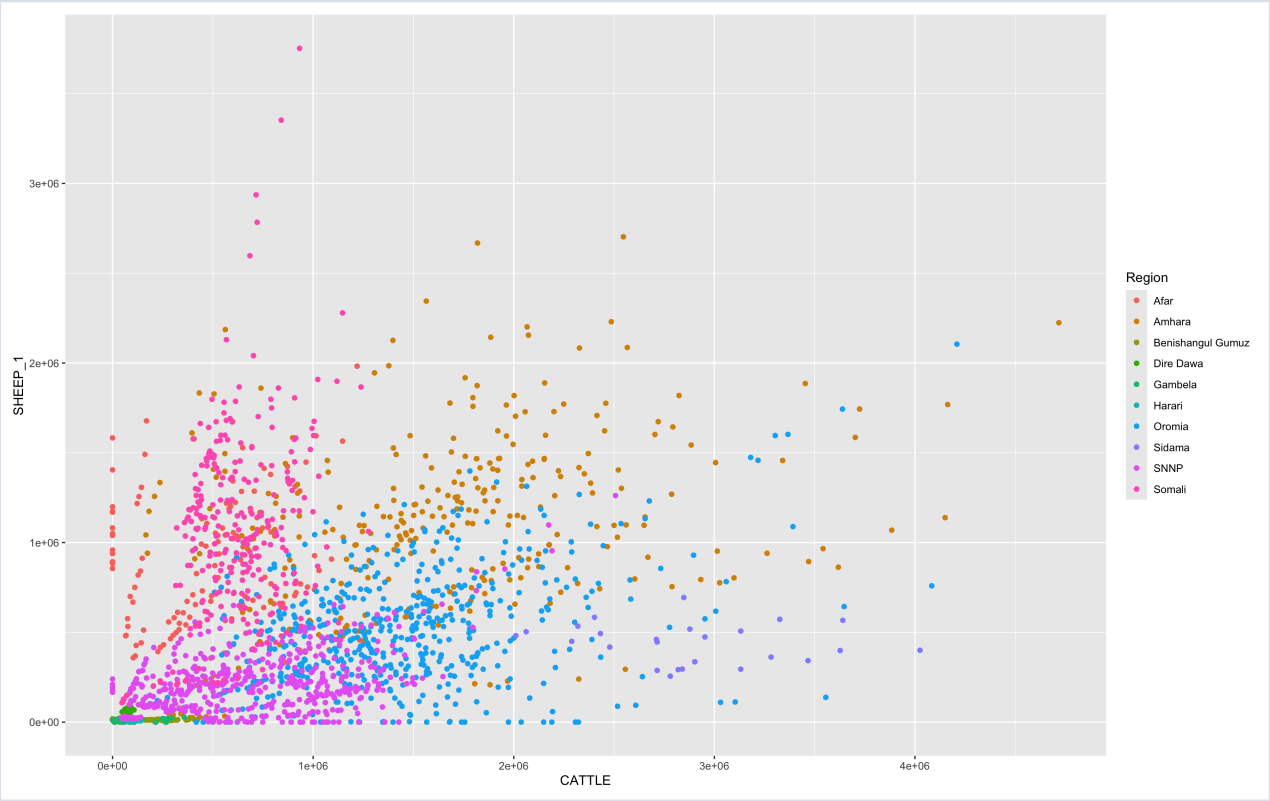
```r
ggplot(livestock, aes(x = CATTLE, y = SHEEP_1)) + geom_point()
```



Scatter plot: colour by a categorical variable

- Map `colour` to Region to see the pattern for each region.

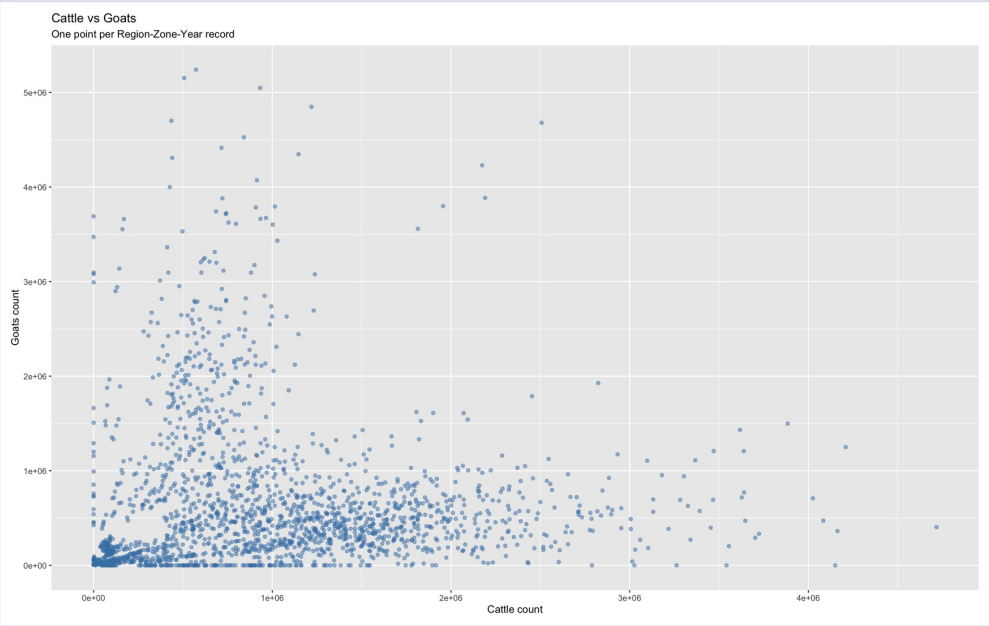
```r
ggplot(livestock, aes(x = CATTLE, y = SHEEP_1, colour = Region)) + geom_point()
```



Scatter plot: titles and labels with `labs()`

- Store the plot in an object, add `labs()`, then `print()` it.

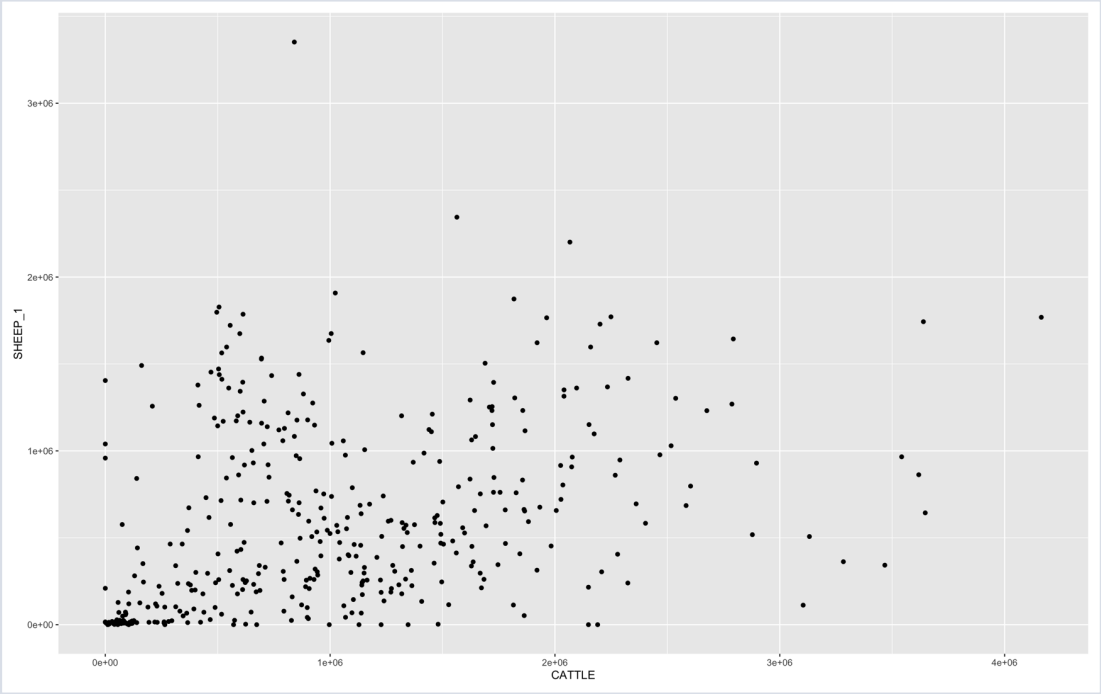
```r
p1 <- ggplot(data6, aes(x = CATTLE, y = GOATS_0)) +
  geom_point(alpha = 0.5, colour = "steelblue") +
  labs(title = "Cattle vs Goats", subtitle = "One point per zone",
       x = "Cattle count", y = "Goats count")
print(p1)
```



Scatter plot: filter, then plot

- A dplyr pipeline can feed straight into `ggplot()` — filter one year, then plot cattle against sheep.

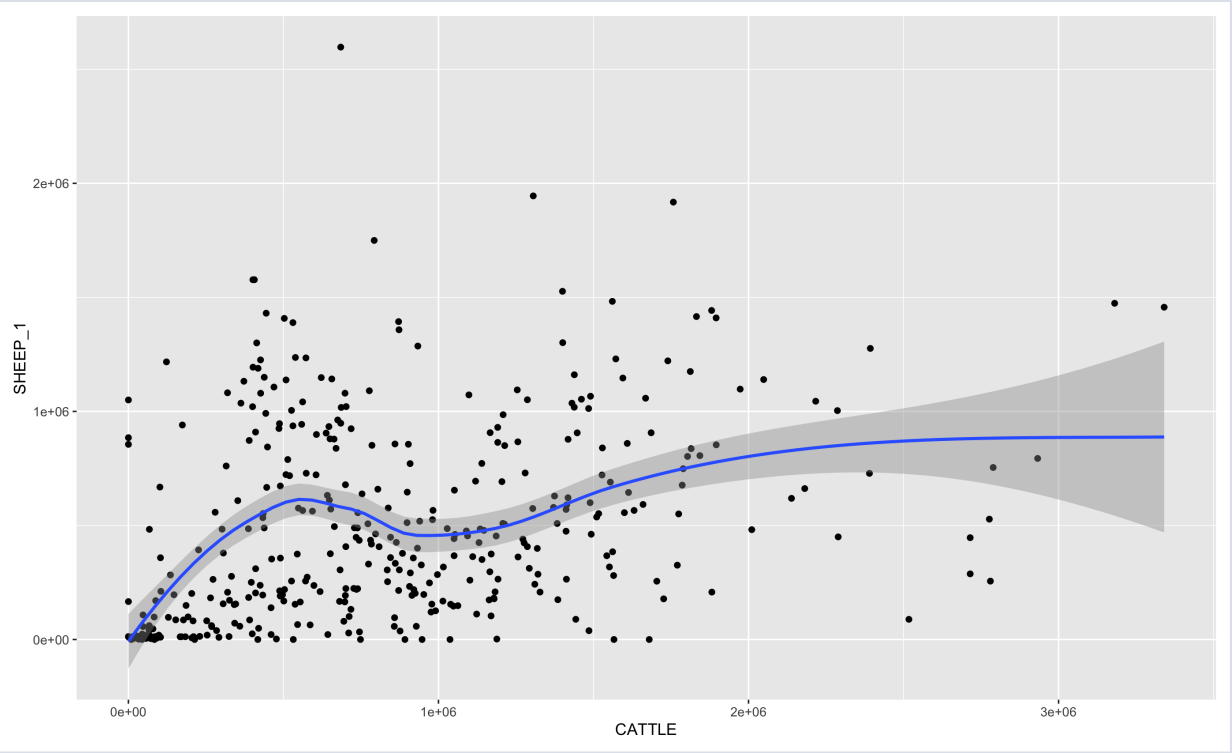
```r
livestock %>% filter(year == 2016) %>%
  ggplot(aes(x = CATTLE, y = SHEEP_1)) + geom_point()
```



Smoothing the scatter: geom_smooth()

- Add a smoothed trend line (with a confidence band) to make the pattern clearer.

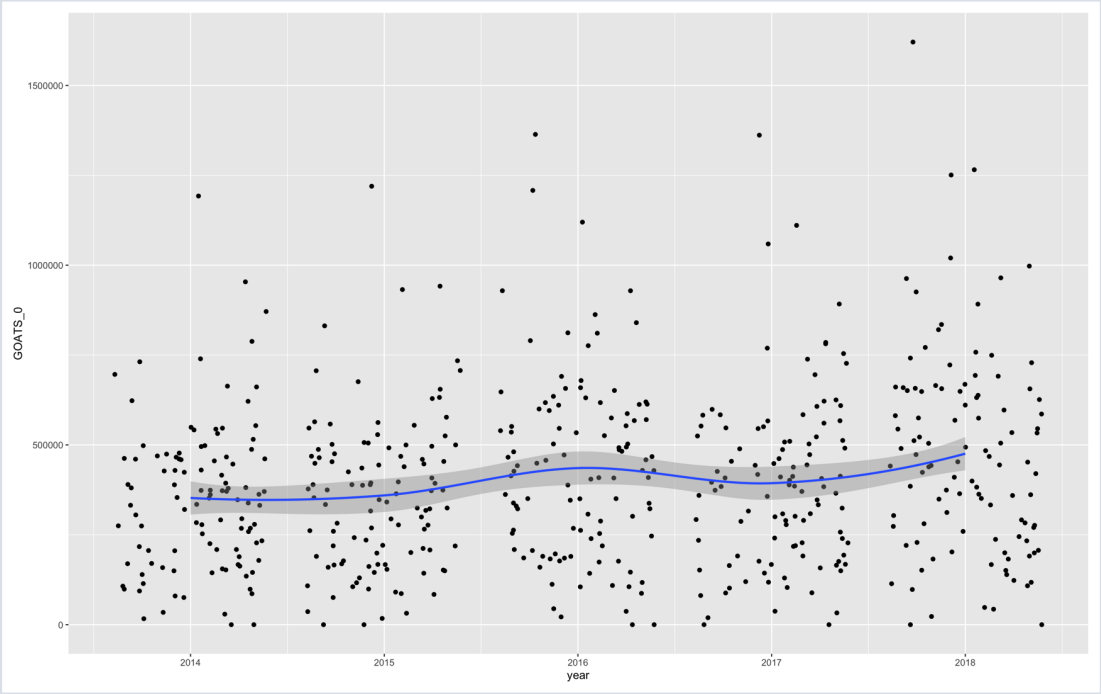
```R
livestock %>% filter(year == 2014) %>%
  ggplot(aes(x = CATTLE, y = SHEEP_1)) + geom_point() + geom_smooth()
```



Scatter plot: geom_jitter()

- Jitter adds a small random shift so that overlapping observations become visible instead of appearing as one point.

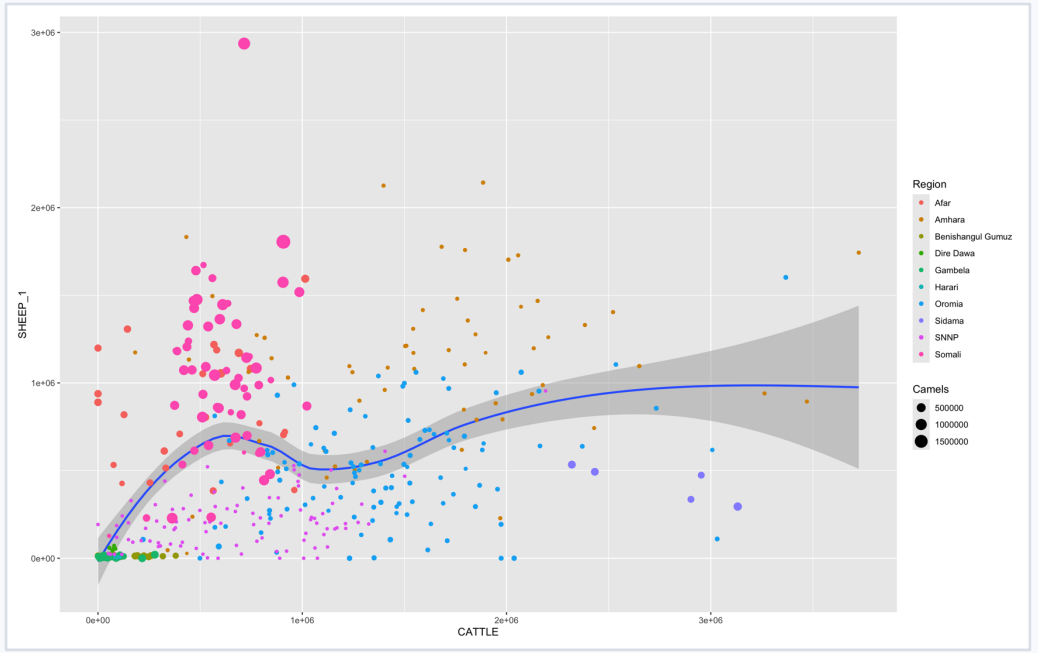
```R
livestock %>% filter(Region == "Oromia") %>%
  ggplot(aes(x = year, y = GOATS_0)) + geom_jitter() + geom_smooth()
```



Scatter plot: adding a third variable with `size`

- Mapping `size` to camels shows the camel population at every point while keeping the cattle–sheep relationship per region.

```r
livestock %>% filter(year == 2017) %>%
  ggplot(aes(x = CATTLE, y = SHEEP_1)) + geom_smooth() +
  geom_point(aes(colour = Region, size = Camels))
```

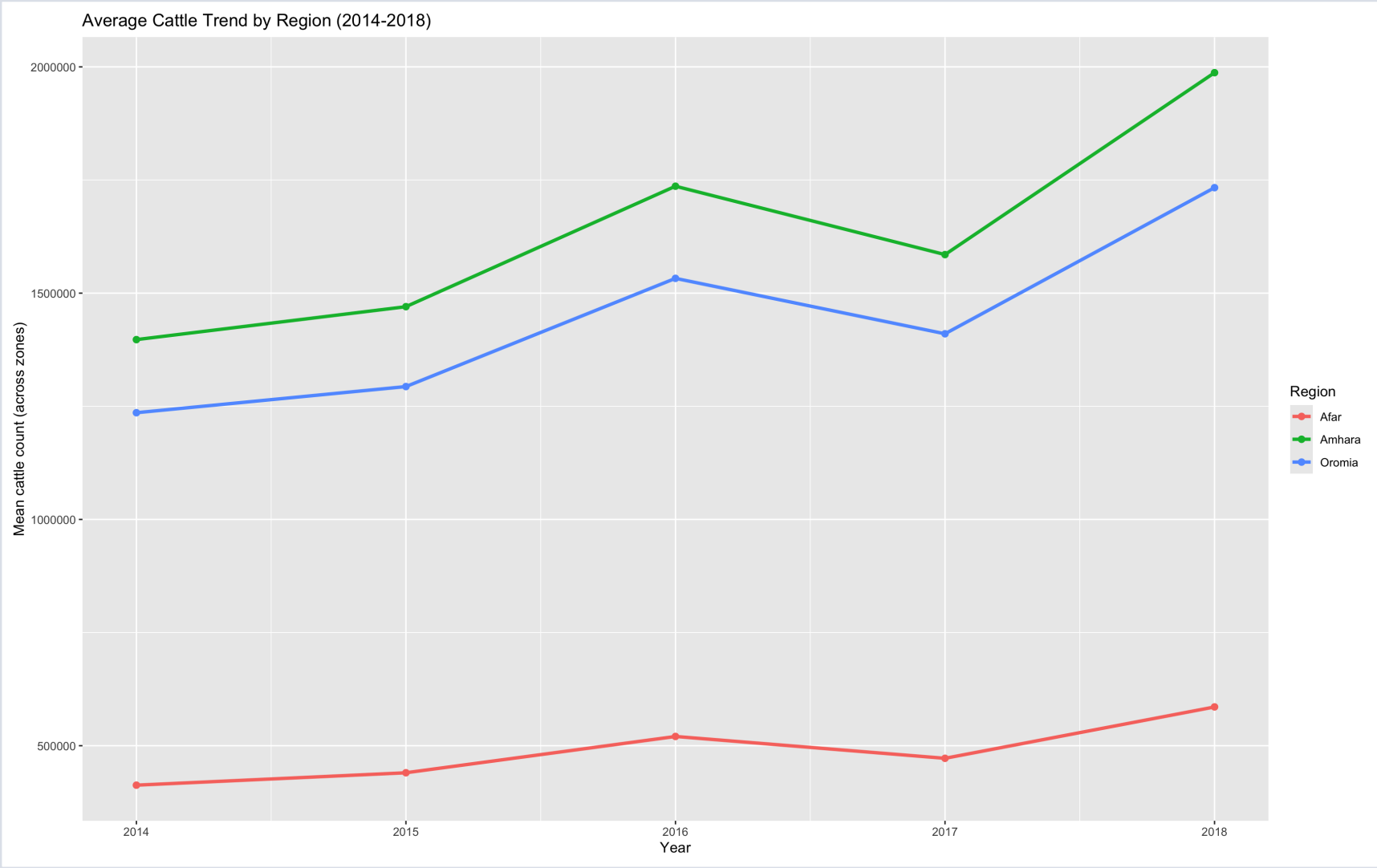


Line chart: geom_line()

- Summarise with dplyr, then map year to x, the summary to y and Region to colour. `geom_line()` connects the points; `geom_point()` marks them.
- Example: average cattle trend for Afar, Amhara and Oromia, 2014–2018.

```R
livestock %>%
  filter(Region %in% c("Afar", "Amhara", "Oromia")) %>%
  group_by(Region, year) %>%
  summarise(Mean_Cattle = mean(CATTLE)) %>%
  ggplot(aes(x = year, y = Mean_Cattle, colour = Region)) +
  geom_line(linewidth = 1.2) + geom_point(size = 2) +
  labs(title = "Average Cattle Trend for three Regions (2014-2018)",
       x = "Year", y = "Mean cattle count")
```

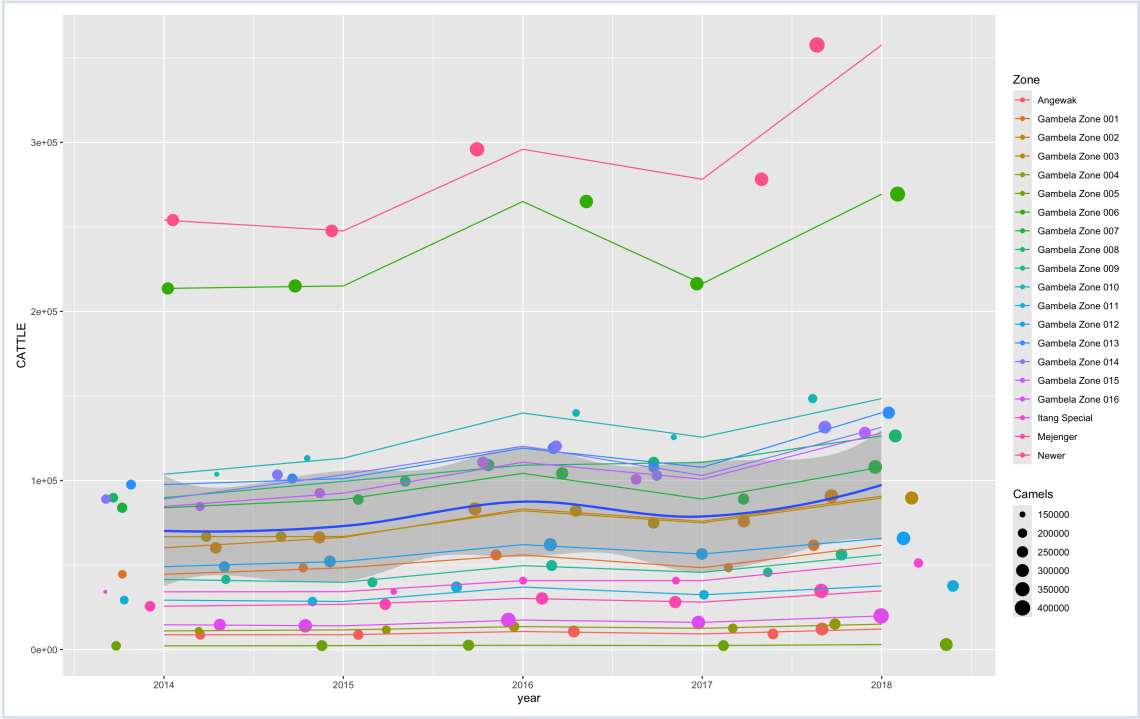
Line chart: average cattle trend for three regions



Line chart: layering jitter, smooth and lines

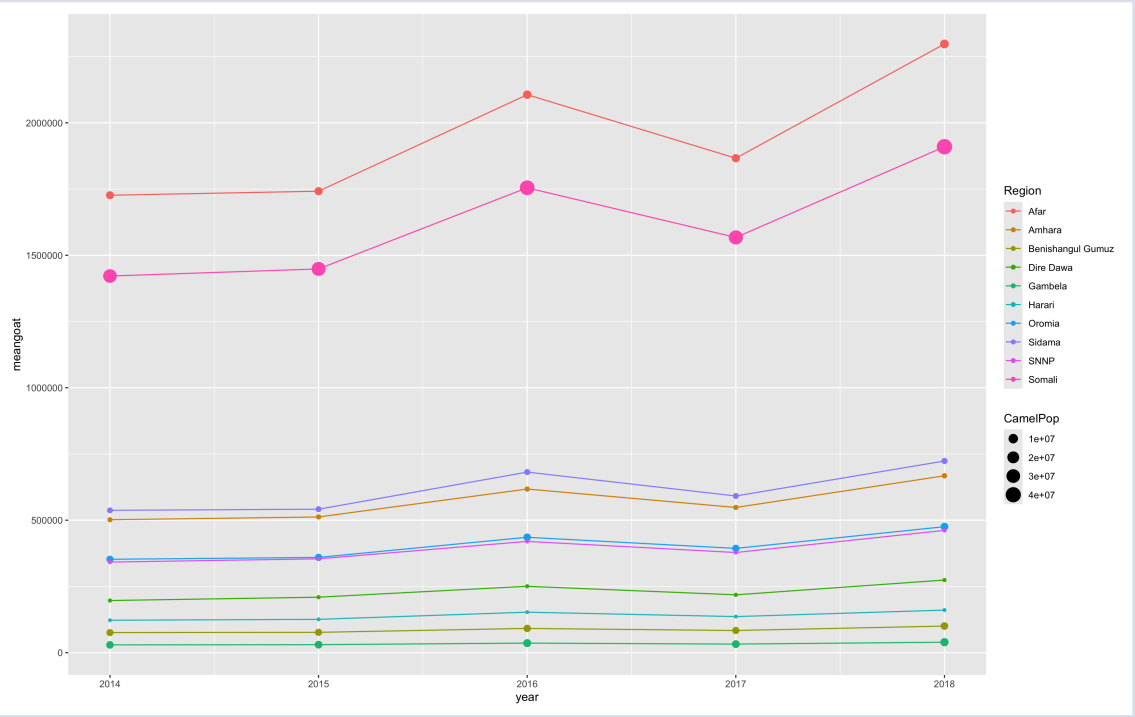
- Example: cattle trend in Gambela by zone, 2014–2018.

```r
livestock %>% filter(Region == "Gambela") %>%
  ggplot(aes(x = year, y = CATTLE)) +
  geom_jitter(aes(colour = Zone, size = Camels)) + geom_smooth() +
  geom_line(aes(colour = Zone))
```



Line chart: mean goats by region, point size = camels


```r
livestock %>% group_by(Region, year) %>%
  summarise(meangoat = mean(GOATS_0), CamelPop = sum(Camels)) %>%
  ggplot(aes(x = year, y = meangoat, colour = Region)) +
  geom_line() + geom_point(aes(size = CamelPop))
```



DISCUSSION

Is this chart easy to read? What would you remove or split into facets to make one message stand out?

TAKE A BREAK

Tea Break

10:30 – 10:45 · 15 minutes

UP NEXT

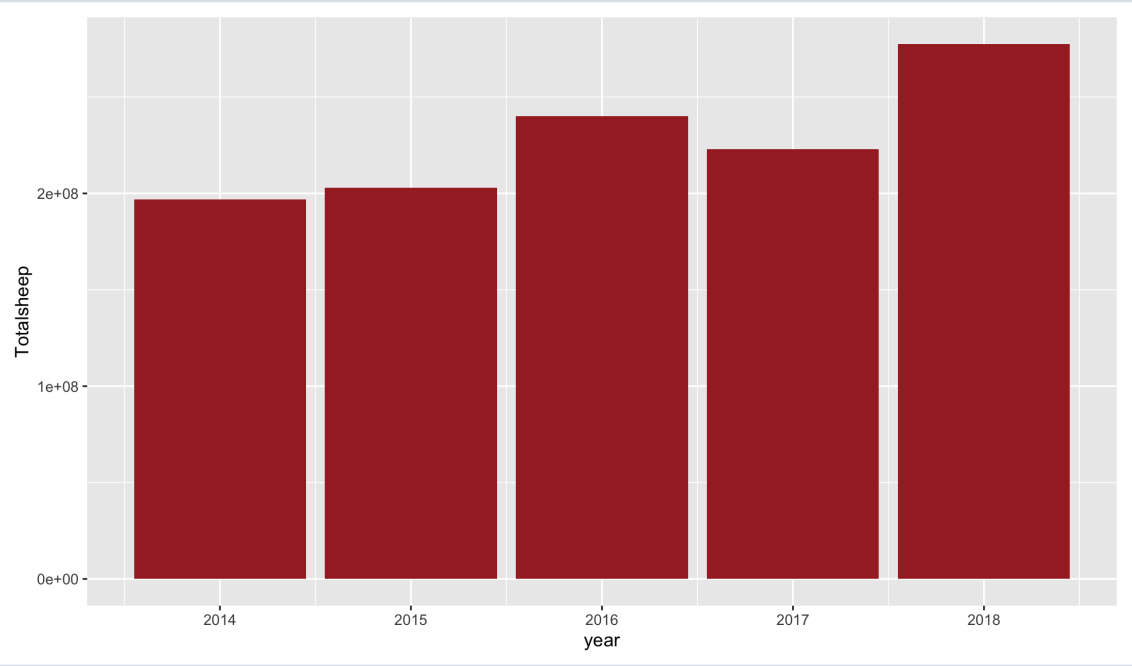
Bar charts · R Markdown and reproducible reports



Bar chart: `geom_col()`

- A column chart shows a value per discrete category. Example: total sheep population at national level per year.

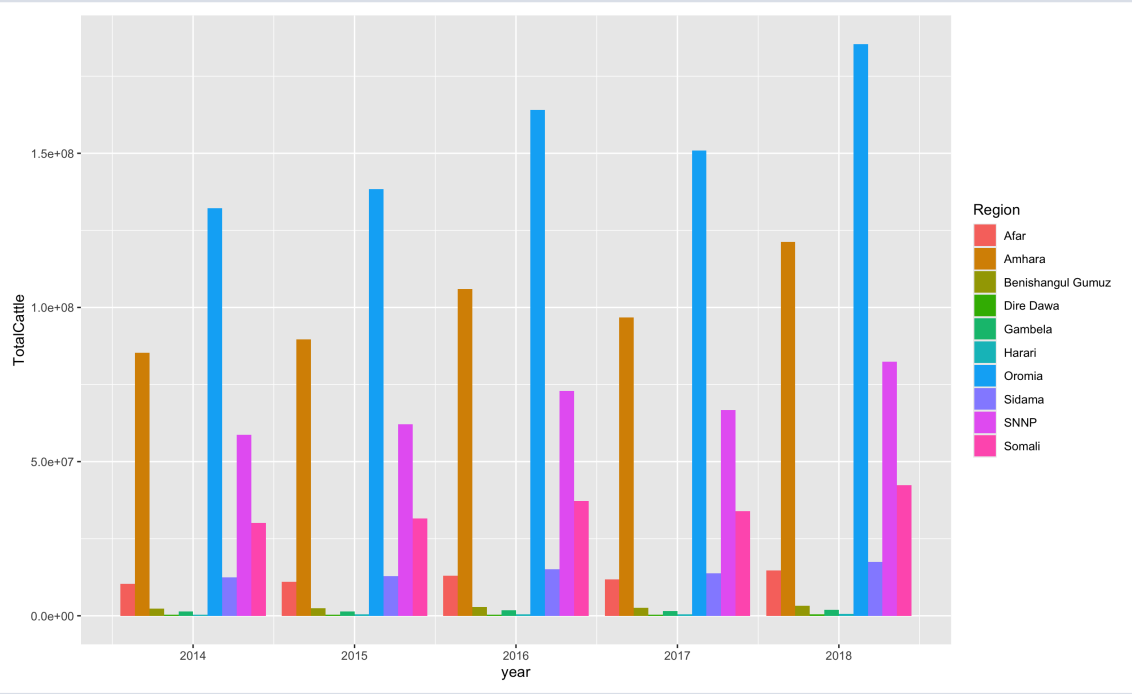
```R
livestock %>% group_by(year) %>%
  summarise(Totalsheep = sum(SHEEP_1)) %>%
  ggplot(aes(x = year, y = Totalsheep)) + geom_col(fill = "blue")
```



Clustered bar chart: `position = "dodge"`

- Compare total cattle for all regions, 2014–2018, with bars side by side.

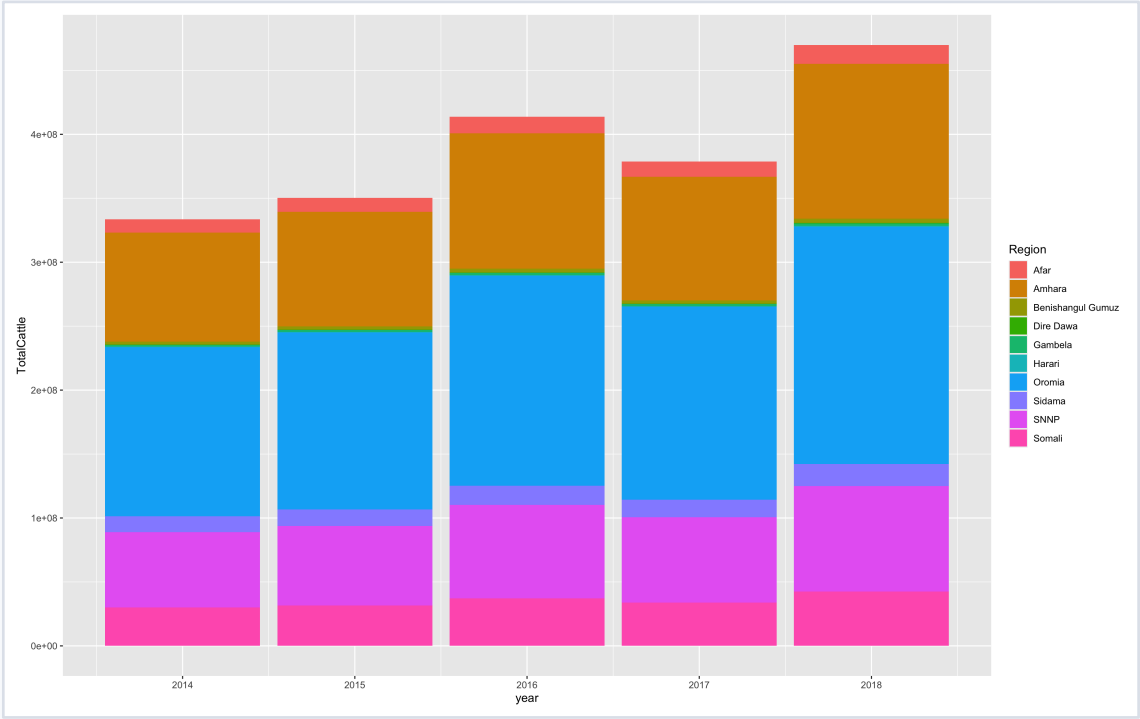
```r
livestock %>% group_by(Region, year) %>%
  summarise(TotalCattle = sum(CATTLE)) %>%
  ggplot(aes(x = year, y = TotalCattle, fill = Region)) +
  geom_col(position = "dodge")
```



Stacked bar chart

- With `fill` mapped to a category and no `position`, `geom_col()` stacks the bars — good for totals and composition.

```r
livestock %>% group_by(Region, year) %>%
  summarise(TotalCattle = sum(CATTLE)) %>%
  ggplot(aes(x = year, y = TotalCattle, fill = Region)) + geom_col()
```



BRAINSTORM

Which ESS chart would you re-draw with ggplot2 first? Name the data, the geom and the aesthetics.



HANDS-ON EXERCISE

25 min

Exercise: three charts from the livestock data

1. Scatter: goats against camels for 2018, coloured by Region, with a title and axis labels.
2. Line: mean sheep per year for your own region compared with the national mean.
3. Bar: total goats per region for the latest year, sorted from largest to smallest (hint: `reorder()`).
4. Stretch: add `facet_wrap(~ Type)` to one of the charts and `theme_minimal()`.

DAY 3 · MODULE 7

07

Reports & Dashboards

From reproducible R Markdown reports to Quarto and Shiny dashboards.

In this module you will

- Write Markdown text and a YAML header
- Use inline code and code-chunk options
- Add tables, plots and tabsets to a report
- Explain how Quarto extends R Markdown
- Build a Quarto dashboard with value boxes and interactive charts

R Markdown and parameterised reporting

- **Markdown** is a lightweight markup language for formatting plain text with simple, readable symbols instead of HTML.
- A document looks almost as clean in raw text as it does once rendered.
- **R Markdown** combines Markdown text with executable R code chunks — the report and the analysis live in one file.

Markdown core components

Headings

`#`, `##`, `###` for levels 1–3.

Emphasis

`*italic*` and `**bold**`.

Lists

`-` for bullets, `1.` for numbered.

Links & images

`[text](url)` and `![alt](path)`.

Code

Backticks for inline code; triple backticks for blocks.

Tables

Built with pipes `|` and dashes `-`.

Markdown example

- This renders as a heading, a paragraph with bold and italic text, a bullet list and a clickable link.

```r
# My Report

This is **bold** and this is *italic*.

- First point
- Second point

[Visit R Markdown](https://rmarkdown.rstudio.com)
```

The YAML header

- **YAML** (*YAML Ain't Markup Language*) is a simple, human-readable format for structured key–value settings.
- In R Markdown it holds **configuration only**: title, author, output format, theme … No code runs in YAML.
- The two `---` lines at the very top of the file **delimit** the metadata block for the rendering engine.

```R
---
title: "R Markdown — Core Concepts Demo"
author: "ESS Data Visualization Training"
output: html_document
---
```

`---`: two jobs for the same symbol

- **At the very top of the file**, surrounding the metadata: the **YAML delimiter**.
- **On its own line in the body**: a **horizontal rule** — a divider line used to separate sections.

```R
Some text here.

---

Some more text after a divider line.
```

YAML example

```yaml
# ================================================================================
# YAML FRONT MATTER (also called the "YAML header")
# --------------------------------------------------------------------------------
# This block, between the two "---" lines, is NOT R code -- it is metadata
# that tells R Markdown / Pandoc how to build the final document.
# Every key below is optional except `output`, but these are the ones you
# will use constantly.
# ================================================================================
title: "R Markdown — Core Concepts Demo"      # document title (shown at the top)
author: "ESS Data Visualization Training"      # author name
date: "`r Sys.Date()`"                         # <- inline R code even works in YAML!
output:
  html_document:                               # output FORMAT (others: pdf_document, word_document)
    toc: true                                  # adds an automatic Table of Contents
    toc_float: true                            # makes the ToC "float" alongside the page as you scroll
    number_sections: true                      # auto-numbers headings (1, 1.1, 1.2, 2, ...)
    theme: cosmo                               # a built-in Bootstrap visual theme
    highlight: tango                           # syntax-highlighting colour scheme for code
params:                                        # PARAMETERS: values you can inject at render time
  species: "Cattle"                            # e.g. rmarkdown::render(..., params = list(species = "Sheep"))
---
```

R Markdown text formatting

- Headings are numbered automatically when `number_sections: true` is set in the YAML header; `##` nests under `#` as 1.1.

```R
# 1. Markdown Text Formatting

- **Bold text** uses double asterisks
- *Italic text* uses single asterisks
- `Inline code` uses backticks
- A [hyperlink](https://rmarkdown.rstudio.com) uses [text](url)
- Bullet lists use - or *; numbered lists use 1., 2., ...

## 1.1 A Second-Level Heading
```

Inline R code

- Embed live R results inside a sentence with `` `r expression` ``.
- **Why it matters:** the numbers re-calculate every time you knit, so the report never goes stale when the data changes.

```R
The dataset has `r nrow(mtcars)` rows and `r ncol(mtcars)` columns,
and today's date is `r Sys.Date()`.
```

Code chunks and chunk options

- A **code chunk** is a fenced block starting with ` ```{r} ` and ending with ` ``` `. The text after `r` is the chunk **label**.

R

````
```{r chunk-options-demo, echo=TRUE, eval=TRUE, fig.cap="A simple example plot"}
#   echo    = TRUE/FALSE  -> show/hide the R code
#   eval    = TRUE/FALSE  -> run the code, or just display it
#   include = TRUE/FALSE  -> show/hide BOTH code and output (still runs)
#   message, warning      -> show/hide messages and warnings
#   fig.width, fig.height -> plot size for this chunk
#   fig.cap               -> caption under the figure
#   cache   = TRUE/FALSE  -> cache slow results between knits
summary(mtcars$mpg)
```
````

Hiding code, keeping results: `echo = FALSE`

- Run the code but do **not** print it — only the output appears. Common in final reports where readers want the result, not the code.
- Objects created in a hidden chunk remain available for inline code.

R

````
```{r hidden-code-example, echo=FALSE}
avg_mpg <- mean(mtcars$mpg)
```

The average mpg computed by the hidden chunk above is `r round(avg_mpg, 1)`.
````

Tables and plots in a report

- `knitr::kable()` turns any data frame into a clean, publication-style table.
- Any ggplot2 (or base R) plot inside a chunk is rendered as an embedded image — no extra steps.

R

````
```{r}
kable(head(mtcars, 5), caption = "First 5 rows of the mtcars dataset")
```

```{r cattle-plot, fig.cap="MPG vs horsepower, coloured by cylinders"}
ggplot(mtcars, aes(x = hp, y = mpg, colour = factor(cyl))) +
  geom_point(size = 3) +
  labs(title = "MPG vs Horsepower", colour = "Cylinders") +
  theme_minimal()
```
````

Tabbed sections: `{.tabset}`

- Adding `{.tabset}` to a heading turns its sub-headings into clickable **tabs** in HTML output — more content in less scroll space.

R

```
# 7. Tabbed Sections {.tabset}

## Tab A: Summary
```{r}
summary(mtcars$hp)
```

Tab B: Histogram
```{r}
hist(mtcars$hp, main = "Horsepower Distribution", col = "steelblue")
```
```

DISCUSSION

Which ESS publication could become an R Markdown report that re-knits itself each survey round? What would be parameterised?

TAKE A BREAK

Lunch Break

12:30 – 13:30 · 60 minutes

UP NEXT

Quarto and Quarto dashboards

Quarto

- **Quarto** is an open-source, next-generation publishing system from Posit (makers of RStudio and R Markdown).
- Think of it as R Markdown's successor: it combines narrative text, code and output in one rendered document —
  - but it is **language-agnostic** (R, Python, Julia, Observable JS) and does not depend on the R package ecosystem to run.

Core components of Quarto

.qmd files

The Quarto equivalent of .Rmd — same Markdown + code-cell structure.

YAML front matter

Same idea as R Markdown, with its own (partly different) options: format:, execute:, code-fold.

Code cells

Fenced blocks `{r} or `{python}; mixed languages side by side in one document.

Quarto CLI

A standalone command-line tool (quarto render) independent of any IDE or R package.

Built-in formats

Dashboards (tabs, cards, value boxes), books, websites, reveal.js presentations, journal templates.

R Markdown versus Quarto

	R Markdown	Quarto
Engine	R's `knitr` / `rmarkdown` packages	Standalone CLI (works even without R installed)
Languages	R-centric (Python/Julia via reticulate, less native)	R, Python, Julia, Observable — natively mixed
Ecosystem	Tied to R package installs	Independent tool; R is just one supported engine
Dashboards	Needs the separate `flexdashboard` package	Native `format: dashboard` built in
Cross-referencing, citations	Possible but more manual	More built-in, unified across formats

How to run Quarto

- **Install the Quarto CLI** (separate from R/RStudio) from *quarto.org/docs/get-started*.
- **Check** the installation in a terminal: `quarto check` — it also reports whether R and Python/Jupyter were found.
- **Save** the file as `quarto_core_concepts_demo.qmd` in your working folder.
- **Render** from the terminal, or in RStudio (2022.07+) open the `.qmd` and click **Render** — the same one-click workflow as knitting an `.Rmd`.
- **Open** the resulting `.html` in any browser.

```R
quarto render quarto_core_concepts_demo.qmd
# -> quarto_core_concepts_demo.html in the same folder
```

What changes from R Markdown to Quarto

YAML

format: instead of output:, hyphenated names (number-sections), a global execute: block, code-fold: true.

Code cells

Quarto's term for chunks; options written as #| option: value inside the cell.

Cross-references

@fig-mpg-hp / @tbl-mtcars — built in and far more automatic.

Callouts

.callout-note, .callout-warning, .callout-tip — Quarto-only.

Tabsets

::: {.panel-tabset} fenced divs replace the {.tabset} heading trick.

Unchanged

Inline code, knitr::kable(), LaTeX maths, params$..., sessionInfo() for reproducibility.

A Quarto dashboard

- A Quarto dashboard is a special output format (`format: dashboard`) built for laying out **multiple charts, tables and KPI cards on one page** in a grid or tab structure — like flexdashboard, but native to Quarto.
- **Core idea:** instead of a document that flows top to bottom, content is arranged into **cards** organised by rows, columns and pages — a control panel rather than a report.

Quarto dashboard building blocks

format: dashboard

In the YAML header — switches Quarto from document mode to dashboard mode.

Headings = layout

becomes a page (tab); ## becomes a row or column within the page.

Cells = cards

Each code cell becomes a self-contained panel with its own border and title.

Value boxes

KPI cards with a number, icon and colour via #| content: valuebox.

Sizing

{width=30%} or {height=200} on a heading controls the space it takes.

Sidebar

An optional .sidebar section for filters or navigation, like Shiny's sidebar.

Walkthrough 1 · the header

- `format: dashboard` turns the document into a dashboard layout.
- `orientation: columns` — top-level `##` headings are read as **columns side by side**, not stacked rows.
- `theme: cosmo` — the same Bootstrap theme system used in regular Quarto documents.

yaml

```yaml
title: "ESS Livestock Dashboard — Advanced (Quarto)"
format:
  dashboard:
    theme: cosmo
    orientation: columns
```

Walkthrough 2 · setup cell and first column

- `#| include: false` hides the setup cell entirely — it only loads packages and data.
- `latest` filters to the most recent year for the cards that need current totals.
- `## Column {width=25%}` creates a narrow first column for the KPI cards.

```r
#| include: false
library(tidyverse)
library(scales)
library(plotly)
livestock <- readRDS("livestock_clean.rds")
region_year_summary <- readRDS("region_year_summary.rds")
latest <- region_year_summary %>% filter(Year == max(Year))
```

Walkthrough 3 · value boxes

- `#| content: valuebox` turns the cell output into a KPI card. The cell must return a list with `icon`, `color` and `value`.
- `icon = "cow"` comes from Bootstrap Icons; `color = "success" / "info" / "warning"` map to green, blue and orange.
- `scales::comma()` formats the number with thousands separators. Repeated three times: Cattle, Sheep, Goats.

```r
#| content: valuebox
#| title: "Total Cattle"
list(icon = "cow", color = "success", value = scales::comma(sum(latest$Cattle)))
```

Walkthrough 4 · the chart column

- `## Column {width=75%}` — the wide second column where the two charts live.
- `### Row` (twice) — inside a column, `###` creates rows, so the charts **stack vertically**.
- `#| title:` becomes the card header. A normal ggplot is wrapped in `ggplotly(p)` to make it **interactive** (hover, zoom).

Chart cell 1

```r
#| title: "National Species Trend"
p <- region_year_summary %>%
  pivot_longer(c(Cattle, Sheep, Goats, Camels), names_to = "Species", values_to = "Cou
  group_by(Year, Species) %>% summarise(Count = sum(Count), .groups = "drop") %>%
  ggplot(aes(Year, Count, color = Species)) + geom_line(linewidth = 1) + geom_point()
  scale_y_continuous(labels = scales::comma) + theme_minimal()
ggplotly(p)
```

Walkthrough 5 · the result

- Chart cell 2 follows the same pattern — a horizontal bar chart of cattle by region for the latest year, also wrapped in `ggplotly()`.
- **End result:** a slim left column with three coloured KPI cards and a wide right column with two interactive charts — all from one linear `.qmd` file where headings double as layout instructions.

Sample livestock dashboard built with Quarto

ESS Livestock Dashboard — Advanced (Quarto)

Total Cattle

568,737,71

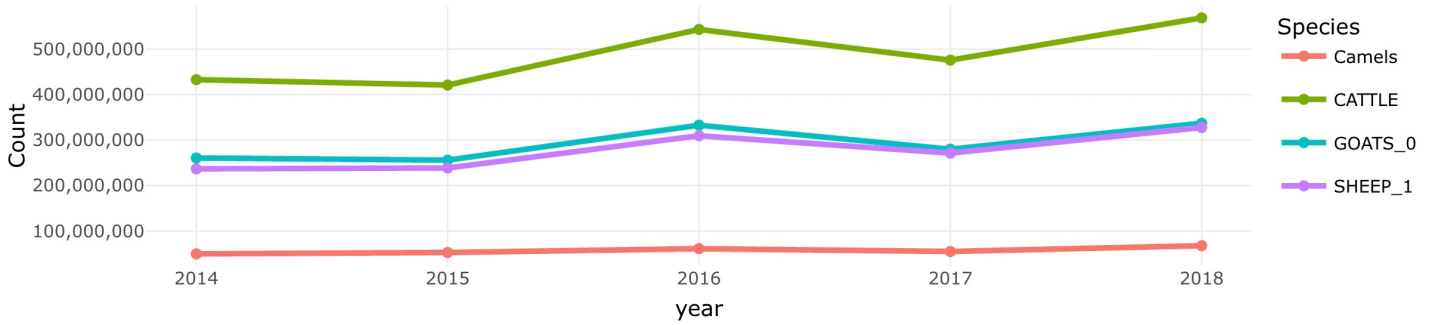
Total Sheep

327,615,37

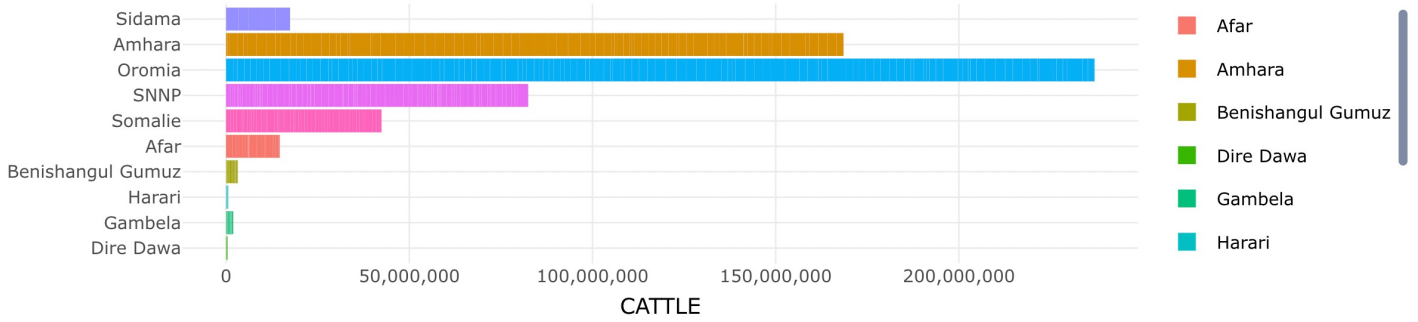
Total Goats

337,104,98

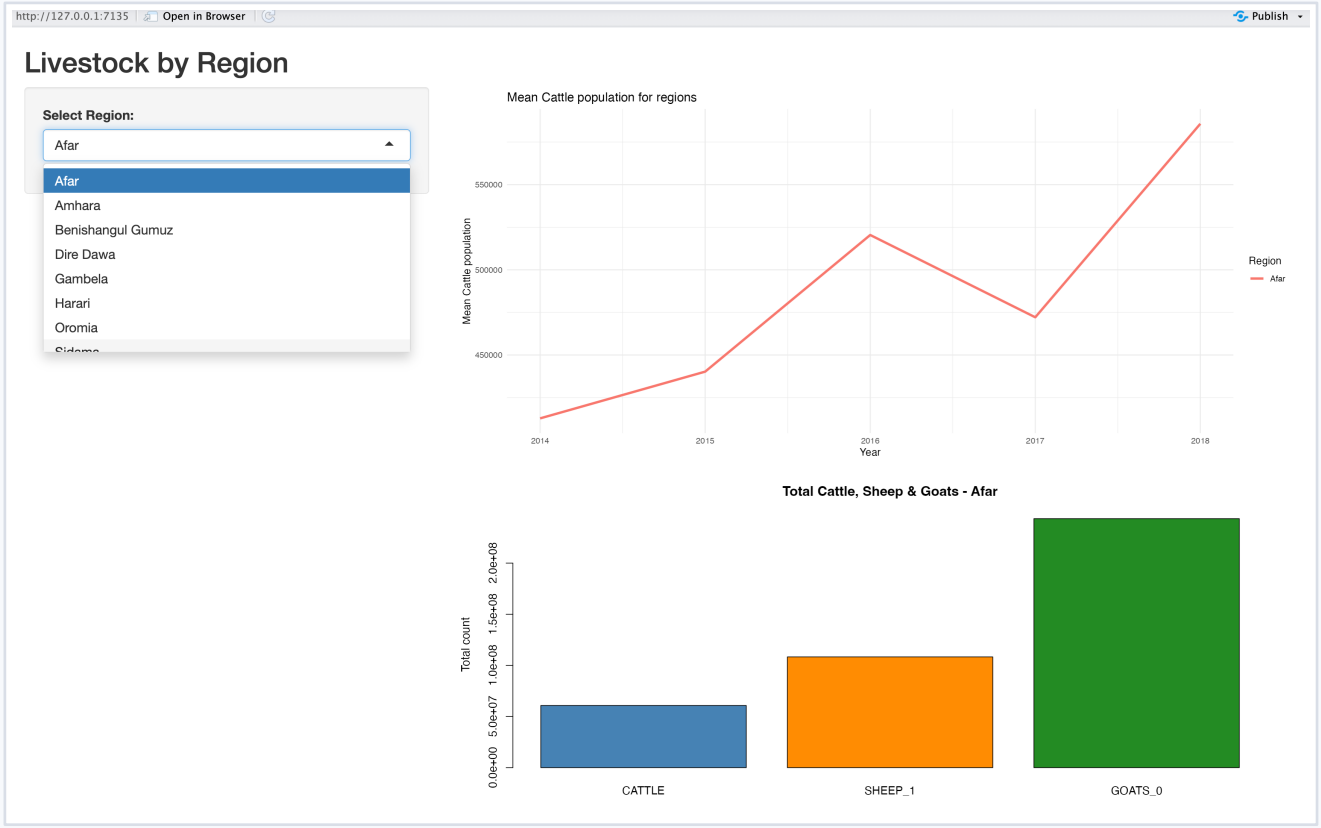
National Species Trend



Cattle by Region (latest year)



Sample livestock dashboard built with Shiny



DISCUSSION

Quarto (re-render on a schedule) or Shiny (live server-side reactivity)? Which fits how ESS publishes indicators today?

TAKE A BREAK

Tea Break

15:00 – 15:15 · 15 minutes

UP NEXT

Hands-on: build your own dashboard





HANDS-ON EXERCISE

60 min

Hands-on: build your own livestock dashboard

1. Start from the walkthrough `.qmd`. Load `MergedLivestock.csv` (created on Day 2) in the setup cell.
2. Create three value boxes — total cattle, sheep and goats for the latest year.
3. Add one `ggplotly()` line chart of the trend over years and one bar chart comparing regions.
4. Stretch: add a second page (`#`) with a `{.panel-tabset}` of tables by Type, or a `.sidebar` with a region selector (Shiny).

How to run it: Pairs work together; facilitators circulate. At 16:00 ask three pairs to render and demo their dashboards for two minutes each.



BRAINSTORM

15 min

From training to practice: dashboards for ESS

1. Which ESS statistical product would benefit most from a Quarto report or dashboard? Who would use it and how often?

2. Which data-preparation steps from Day 2 would have to run automatically before each render?

3. What support (data access, hosting, review) would you need to publish a dashboard internally?

How to run it: Capture concrete ideas on the flipchart; they can feed a follow-up plan with the DAVT team.



QUICK CHECK

25 min

Post-assessment: how far have we come?

1. Fifteen open questions on the same topics as Friday's pre-assessment.

2. Answer individually on the response sheet (Q1–Q15) in one or two sentences, or with a line of code. Not graded; no discussion until everyone has finished.

3. We will compare the group's pre- and post-assessment answers before we close.

How to run it: Collect sheets, then reveal the model answers (hidden slides at the end of the deck) and discuss questions most people found hard.

Post-assessment

Q1 Explain to a new colleague why we write R scripts in RStudio instead of typing in the console. Mention two RStudio panes you now use.

Q2 What is the difference between `typeof(5)` and `typeof(5L)`? Why does `as.integer(78.5)` give 78?

Q3 A survey codes geography as 1–4. Write the R code that turns it into a factor with the labels Lowland, Midland, Highland, Mountainous.

Post-assessment

Q4 Describe what `str(livestock)` and `summary(livestock)` show you, and what you would check first in each output.

Q5 `mean(livestock$HORSES)` returns NA. Explain why and write the corrected call.

Q6 Write base-R code that keeps only Oromiya records for 2018 with the columns Zone, CATTLE and year.

Post-assessment

Q7 Rewrite the same selection using dplyr verbs and the pipe. Which version do you prefer, and why?

Q8 Using `group_by()` and `summarise()`, how would you compute the mean sheep population per Region and year?

Q9 Two files share a `zone` column but spell some zones differently. Describe the steps you would follow before and during `merge()`.

Post-assessment

Q10 Explain the roles of `ggplot()`, `aes()` and a `geom_*()` layer. Sketch (in code) a scatter plot of cattle against sheep coloured by Region.

Q11 What is the difference between `geom_col(position = 'dodge')` and the default `geom_col()` when `fill = Region`? When would you use each?

Q12 What is the purpose of the YAML header in an R Markdown or Quarto file? Give two settings you might put in it.

Post-assessment

Q13 Explain the difference between `echo = FALSE` and `include = FALSE` in a code chunk.

Q14 In a Quarto dashboard, how do headings control the layout, and what turns a code cell into a KPI value box?

Q15 Describe one ESS indicator you would put on a dashboard: the audience, the filters, the chart type and the data preparation needed.

Key takeaways from the three days

1 R fundamentals

Types, objects, vectors, data frames, conditions, loops and functions are the grammar of everything that follows.

2 Data preparation

Read, inspect, subset, clean and combine datasets — in base R and with dplyr — using reproducible scripts.

3 Charts that tell a story

ggplot2 builds charts layer by layer; choose the geom for the question.

4 Reports & dashboards

R Markdown and Quarto turn analysis into re-renderable reports and dashboards.



DISCUSSION

Return to Friday's flipchart: which of the repetitive tasks you listed can you now automate?

Thank you

You now have a complete R toolkit for data analysis, visualisation and dashboards.



- Keep practising with your own datasets — 20 minutes a day beats one long session a month.
- Share your scripts and dashboards with colleagues; peer review speeds up learning.
- Reach out to the DAVT team and facilitators for follow-up support and code reviews.
- Please complete the training evaluation form before you leave.